

Foundations of Embodied AI and Robotics

Machine Learning, Perception, Planning,
Estimation, and Control

ROMAN GRIGORII

Ph.D., Robotics and Human–Machine Interaction
Senior Software Engineer, Robotics

Preface

Robotics is entering a period of unusually rapid change. For much of its history, capable robotic systems were built by decomposing intelligence into carefully engineered components: perception estimated the state of the world, planner generated trajectories, controllers converted those trajectories into forces and torques, and task logic coordinated the system as a whole. This approach remains fundamental to modern robotics, but it is increasingly being complemented by large learned models capable of connecting perception, language, reasoning, and action within a common architecture.

By 2026, the frontier of the field has moved substantially toward *generalist embodied intelligence*. Vision–language–action models can condition robot behavior directly on images, language instructions, proprioceptive state, and other observations. Transformer-based policies, diffusion and flow-based action generators, and large-scale imitation learning are increasingly being used to produce continuous robot behavior rather than merely semantic predictions.

At the same time, robotics remains fundamentally different from purely digital machine learning. A language model can generate an incorrect token with little consequence. A robot acts through a physical body whose motion is constrained by geometry, dynamics, contact, friction, actuator limits, latency, uncertainty, and safety. A successful embodied system must therefore connect modern learning methods to the mathematical and engineering foundations of robotics.

Several problems remain especially important. Robot data are expensive to collect, and available datasets remain small compared with those used to train language and vision models. Human video, teleoperation, simulation, synthetic data, and autonomous experience are consequently being combined to increase scale and diversity. Transferring knowledge between different robot embodiments remains difficult. Long-horizon reliability, recovery from mistakes, manipulation under uncertainty, contact-rich interaction, and generalization to unfamiliar environments remain substantially harder than performing short demonstrations under controlled conditions.

The emerging picture is therefore not one in which classical robotics has been replaced by machine learning. Instead, the field is becoming a synthesis of several disciplines:

robotics+machine learning+computer vision+control+probabilistic estimation+large-scale systems.

Understanding modern embodied AI requires fluency across all of them. I wrote a collection of chapters that I personally use to update myself on concepts quickly, whether for work or for fun, and I hope it can be useful for you as well.

1 Core Machine Learning Concepts

1.1 Backpropagation

Backpropagation efficiently computes gradients of a scalar objective with respect to parameters in a computational graph by recursively applying the chain rule from outputs toward inputs.

For a simple composition,

$$z = Wx, \quad y = \sigma(z), \quad L = \ell(y, t),$$

the gradient with respect to the weights is

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial W}.$$

More generally, if $y = f(x)$,

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}.$$

1.2 Automatic Differentiation (Autograd)

Autograd is the software machinery that records operations in a computational graph and automatically computes derivatives.

Neural-network frameworks typically use reverse-mode automatic differentiation because the loss is scalar while the number of parameters is large.

Forward Pass $\rightarrow$ Loss $\rightarrow$ Backpropagation / Autograd $\rightarrow$ Gradients $\rightarrow$ Optimizer $\rightarrow$ Updated Parameters

Backpropagation describes the gradient-computation procedure, while autograd is the system that implements differentiation automatically.

1.3 Optimizer

An optimizer updates model parameters using gradients in order to minimize an objective.

The basic gradient-descent update is

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t),$$

where η is the learning rate.

Common optimizers include:

- Stochastic Gradient Descent (SGD)
- SGD with Momentum
- Adam
- RMSProp

1.4 Normalization

Normalization rescales or recenters inputs, features, or activations to control their scale and distribution, improving numerical conditioning and optimization.

A common normalization operation is

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

For Layer Normalization, given $x \in \mathbb{R}^d$,

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j,$$
$$\sigma^2 = \frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2,$$

and

$$\hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

A learned scale and shift are then applied:

$$y_j = \gamma_j \hat{x}_j + \beta_j.$$

BatchNorm and LayerNorm primarily differ in which dimensions are used to compute μ and σ^2 .

1.5 Initialization

Initialization determines parameter values before training begins.

A good initialization attempts to preserve activation and gradient variance as signals propagate through the network.

For

$$y = Wx,$$

we ideally want

$$\text{Var}(y) \approx \text{Var}(x).$$

If

$$\text{Var}(y) \gg \text{Var}(x),$$

activations may explode, while

$$\text{Var}(y) \ll \text{Var}(x)$$

may cause vanishing activations and gradients.

Common initialization schemes include:

- Xavier / Glorot initialization
- He / Kaiming initialization

1.6 Regularization

Regularization biases learning toward solutions expected to generalize better rather than simply minimizing training error.

A common formulation is

$$\min_{\theta} [L_{\text{data}}(\theta) + \lambda R(\theta)].$$

For L_2 regularization,

$$R(\theta) = \|\theta\|_2^2.$$

Examples include:

- Weight decay
- Dropout
- Data augmentation
- Early stopping
- Explicit L_1 or L_2 penalties

Regularization is therefore broader than simply adding a penalty term to the loss.

1.7 Loss and Objective Functions

A loss is a scalar-valued function measuring disagreement between a model prediction and its target.

For an individual example,

$$\ell(\hat{y}_i, y_i)$$

is the per-example loss.

The empirical training objective is typically

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$

With regularization,

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i) + \lambda R(\theta).$$

Common losses include:

- Mean Squared Error (MSE)
- Cross-Entropy Loss
- Contrastive Loss
- Hinge Loss

1.8 Embeddings

An embedding maps an object into a continuous vector representation:

$$e : \mathcal{X} \rightarrow \mathbb{R}^d.$$

For example,

$$\text{“cat”} \rightarrow [0.17 \quad -0.82 \quad 0.31 \quad \dots].$$

The goal is for useful semantic or structural relationships to be represented geometrically in the embedding space.

A common similarity measure is cosine similarity:

$$\text{sim}(a, b) = \frac{a^\top b}{\|a\|_2 \|b\|_2}.$$

2 Transformer Fundamentals

Transformers are sequence-processing architectures built primarily around the attention mechanism. Unlike recurrent neural networks, which propagate information sequentially through a hidden state, Transformers allow elements of a sequence to interact directly through learned pairwise attention scores. This property makes them highly parallelizable during training and allows them to represent long-range dependencies efficiently.

The central operation is *scaled dot-product attention*,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where Q , K , and V denote the query, key, and value matrices, respectively. Most of the important components of a Transformer can be understood as mechanisms for constructing these matrices, modifying the attention scores, or efficiently evaluating this operation.

2.1 Queries, Keys, and Values

Consider a sequence of n input representations, each having dimension d_{model} . Stacking the representations row-wise gives

$$X \in \mathbb{R}^{n \times d_{\text{model}}}.$$

A Transformer attention layer learns three linear projections,

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

The query, key, and value matrices are obtained from

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Therefore,

$$Q, K \in \mathbb{R}^{n \times d_k}, \quad V \in \mathbb{R}^{n \times d_v}.$$

For token i , the query vector q_i describes the information that the token attempts to retrieve from the sequence. For token j , the key vector k_j determines how compatible that token is with a particular query, while the value vector v_j contains the information contributed by token j if it receives substantial attention.

The compatibility between query i and key j is measured using a dot product,

$$q_i^T k_j.$$

After scaling, the corresponding attention score becomes

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}.$$

For a fixed query i , the scores are normalized across all candidate keys using the softmax function,

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{m=1}^n \exp(s_{im})}.$$

The output representation associated with token i is then

$$y_i = \sum_{j=1}^n a_{ij} v_j.$$

Attention can therefore be interpreted as a content-dependent weighted combination of value vectors. The weights depend on the compatibility between a query and the keys associated with the available information.

For example, consider the sentence

The robot grabbed the cup because it was nearby.

When constructing the representation of the word ‘it,’ an attention head may assign a large weight to the token ‘cup.’ Information represented by the value vector corresponding to ‘cup’ can then influence the contextual representation of ‘it.’

2.2 Scaled Dot-Product Attention

The factor $1/\sqrt{d_k}$ plays an important numerical role. Consider the unscaled dot product

$$q^T k = \sum_{r=1}^{d_k} q_r k_r.$$

If the components of q and k are approximately independent, zero-mean random variables with comparable variance, the variance of the dot product grows approximately linearly with d_k ,

$$\text{Var}(q^T k) \propto d_k.$$

Consequently, high-dimensional query and key vectors can produce logits with large magnitude. When these logits are passed through softmax, the resulting probability distribution can become strongly saturated, assigning nearly all probability to a small number of entries. Saturation produces small gradients and can make optimization more difficult.

Dividing by $\sqrt{d_k}$ approximately compensates for this growth:

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d_k}}.$$

The resulting operation is therefore known as *scaled dot-product attention*.

2.3 Self-Attention

In self-attention, queries, keys, and values are derived from the same sequence:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

The attention matrix is

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right).$$

Since

$$QK^T \in \mathbb{R}^{n \times n},$$

the attention matrix satisfies

$$A \in \mathbb{R}^{n \times n}.$$

The element A_{ij} specifies how strongly token i , acting as a query, attends to token j , acting as a key.

The layer output is

$$Y = AV.$$

Thus, every output token can contain information aggregated from every input token. This global interaction distinguishes self-attention from operations such as small convolution kernels, whose receptive fields are initially local.

Consider the sequence

The robot picked up the red box.

Different attention interactions may connect ‘red’ with ‘box,’ ‘picked’ with ‘robot,’ or other contextually useful relationships. These interactions are learned from data rather than specified explicitly.

2.4 Cross-Attention

Cross-attention extends the same mechanism to two different sources of information.

Let

$$X_q$$

denote a query-side representation and

$$X_c$$

denote a context representation. Queries are obtained from the first source,

$$Q = X_q W_Q,$$

while keys and values are obtained from the context,

$$K = X_c W_K, \quad V = X_c W_V.$$

The attention calculation remains

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

The distinction between self-attention and cross-attention is therefore determined by the source of the query, key, and value representations.

In self-attention,

$$Q, K, V$$

all originate from the same representation. In cross-attention, the queries originate from one source while the keys and values originate from another.

Cross-attention is particularly important in multimodal systems. Suppose a robot receives the language instruction

Pick up the red cup.

Language features may provide query vectors,

$$Q_{\text{text}},$$

while image features provide

$$K_{\text{vision}}, \quad V_{\text{vision}}.$$

A query associated with the phrase “red cup” can then assign high attention to visual features corresponding to objects whose appearance is consistent with that description. Cross-attention therefore provides a mechanism for selectively exchanging information between modalities.

2.5 Causal Attention

Autoregressive models factorize a sequence probability as

$$P(t_1, \dots, t_n) = \prod_{i=1}^n P(t_i \mid t_1, \dots, t_{i-1}).$$

When predicting token t_i , the model must not have access to tokens $t_{i+1}, t_{i+2}, \dots$, since those tokens are future information.

This constraint is implemented using a causal attention mask,

$$M_{ij} = \begin{cases} 0, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

The attention matrix becomes

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right).$$

For a sequence of four tokens, the allowed attention structure has the form

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

The first token can attend only to itself. The second token may attend to the first two positions, and the third token may attend to the first three.

The value $-\infty$ is used because

$$e^{-\infty} = 0.$$

Consequently, masked entries receive zero probability after the softmax operation. Causal masking therefore enforces the information-flow constraint required for autoregressive prediction.

2.6 Positional Information

The attention operation itself does not contain an inherent representation of sequence order. Without positional information, the tokens in

dog bites man

and

man bites dog

contain the same lexical elements despite having different meanings.

Transformers therefore require an explicit mechanism for representing token position.

2.6.1 Sinusoidal Positional Encoding

The original Transformer introduced deterministic sinusoidal positional encodings. For position pos and embedding dimension index i ,

$$PE(pos, 2i) = \sin \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right),$$

and

$$PE(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right).$$

If e_i is a token embedding and p_i is its positional representation, the Transformer receives

$$x_i = e_i + p_i.$$

The resulting representation contains information about both token identity and token position.

2.6.2 Rotary Positional Embeddings

Many modern Transformers use *Rotary Positional Embeddings* (RoPE). Rather than adding a positional vector to the token embedding, RoPE applies position-dependent rotations to query and key vectors.

Conceptually,

$$q'_i = R(i)q_i,$$

and

$$k'_j = R(j)k_j,$$

where $R(i)$ is a rotation determined by position i .

Attention then depends on

$$(q'_i)^T k'_j.$$

The algebraic structure of the rotations causes this dot product to depend naturally on relative positional relationships such as $i - j$. RoPE therefore introduces position directly into the query–key compatibility calculation.

2.7 Multi-Head Attention

A Transformer normally performs several attention operations in parallel rather than using a single query–key–value projection.

Suppose there are H attention heads. For head h ,

$$Q_h = XW_Q^{(h)},$$

$$K_h = XW_K^{(h)},$$

and

$$V_h = XW_V^{(h)}.$$

Each head computes

$$Z_h = \text{Attention}(Q_h, K_h, V_h).$$

The head outputs are concatenated,

$$Z = \text{Concat}(Z_1, \dots, Z_H),$$

and transformed using a final learned projection,

$$Y = ZW_O.$$

Thus,

$$\text{MultiHead}(X) = \text{Concat}(Z_1, \dots, Z_H)W_O.$$

If the model dimension is divided evenly across the heads, then the dimension of each head is approximately

$$d_h = \frac{d_{\text{model}}}{H}.$$

The use of multiple heads allows the model to represent several interaction patterns simultaneously. Some heads may place greater emphasis on local context, while others may capture long-range dependencies, positional relationships, or semantic correspondences. These interpretations should not, however, be treated as strict assignments; individual attention heads need not have simple human-interpretable roles.

2.8 Computational Complexity of Attention

Full attention compares every query with every key. The score matrix is

$$QK^T \in \mathbb{R}^{n \times n}.$$

Consequently, the size of the attention matrix grows quadratically with sequence length:

$$O(n^2).$$

If the sequence length doubles from n to $2n$, the number of pairwise attention scores grows from

$$n^2$$

to

$$(2n)^2 = 4n^2.$$

This quadratic dependence is one of the principal computational challenges associated with long-context Transformers.

The full computational cost also depends on the hidden dimensions and the feed-forward layers. Nevertheless, the n^2 attention term becomes particularly important when sequence lengths are large.

2.9 KV Caching

KV caching is an inference optimization used in autoregressive Transformers. During token-by-token generation, representations corresponding to previously generated tokens do not need to be recomputed from scratch.

Suppose a model has processed the sequence

The robot picked up the cup

and must generate another token. At each Transformer layer, keys and values have already been computed for the previous tokens:

$$K_{\text{cache}} = [K_1, K_2, \dots, K_t],$$

$$V_{\text{cache}} = [V_1, V_2, \dots, V_t].$$

When token $t + 1$ is introduced, only its new query, key, and value need to be computed:

$$q_{t+1}, \quad k_{t+1}, \quad v_{t+1}.$$

The new key and value are appended to the cache,

$$K_{\text{cache}} \leftarrow [K_{\text{cache}}, k_{t+1}],$$

$$V_{\text{cache}} \leftarrow [V_{\text{cache}}, v_{t+1}].$$

The new query then attends to all previously cached keys and values:

$$\text{Attention}(q_{t+1}, K_{\text{cache}}, V_{\text{cache}}).$$

KV caching therefore trades memory for reduced repeated computation.

A rough scaling law for the number of cached scalar values is

$$O(2BLTH_{kv}d_h),$$

where

- B is the batch size,
- L is the number of Transformer layers,
- T is the cached sequence length,
- H_{kv} is the number of key/value heads,
- d_h is the dimensionality of each head.

The factor of 2 accounts for storing both keys and values.

The actual memory requirement additionally depends on numerical precision. For example, FP16 representations require approximately two bytes per stored scalar. KV-cache storage is therefore an important memory constraint in long-context language-model inference.

2.10 Multi-Head, Multi-Query, and Grouped-Query Attention

In conventional multi-head attention, each attention head has its own query, key, and value projections. If there are H query heads, there are typically also H key and value heads.

Because only keys and values must be retained in the KV cache, inference memory can be reduced by sharing them across multiple query heads.

In *Multi-Query Attention* (MQA), all query heads share a single set of keys and values:

$$H_{kv} = 1.$$

This greatly reduces KV-cache memory.

In *Grouped-Query Attention* (GQA), groups of query heads share key and value heads:

$$1 < H_{kv} < H.$$

GQA provides a compromise between conventional multi-head attention and MQA. It reduces cache size while retaining more key/value projections than a single shared pair.

Since KV-cache memory is approximately proportional to H_{kv} , reducing the number of key/value heads can substantially reduce memory requirements during autoregressive inference.

2.11 Tokenization and Embeddings

A Transformer language model does not operate directly on raw character strings. Before entering the neural network, text is converted into a sequence of discrete token identifiers.

Consider the sentence

Transformers are powerful.

A tokenizer may divide it into subword units conceptually similar to

["Transform", "ers", " are", " powerful", "."].

Each token is assigned an integer identifier,

$$[t_1, t_2, \dots, t_n] \rightarrow [i_1, i_2, \dots, i_n].$$

The exact decomposition and token IDs depend on the tokenizer.

The model contains an embedding matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}},$$

where $|\mathcal{V}|$ is the vocabulary size. If a token has identifier i , its embedding is obtained by indexing the matrix:

$$x = E_i.$$

The input pipeline is therefore

raw text $\rightarrow$ tokens $\rightarrow$ token IDs $\rightarrow$ continuous embeddings.

Most modern language models use subword tokenization. Word-level tokenization produces relatively short sequences but requires very large vocabularies and handles unseen words poorly. Character-level tokenization avoids unknown words but produces much longer sequences. Subword tokenization provides a compromise by allowing common pieces of words to receive their own tokens.

For example, the word

electromechanical

might be represented using pieces conceptually resembling

electro + mechan + ical.

The exact decomposition is determined by the tokenizer vocabulary and segmentation algorithm.

2.12 From Input Tokens to Next-Token Prediction

A decoder-only Transformer performs autoregressive next-token prediction. Consider a prompt containing tokens

$$t_1, t_2, \dots, t_n.$$

The computation can be decomposed into several stages.

First, tokenization maps text to integer identifiers. Each identifier is then converted into an embedding,

$$t_i \rightarrow e_i \in \mathbb{R}^{d_{\text{model}}}.$$

Positional information is incorporated to produce an initial sequence representation,

$$e_i \rightarrow x_i.$$

Within each attention layer, the representation is projected into queries, keys, and values,

$$X \rightarrow Q, K, V.$$

For a causal decoder, attention is evaluated using

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right),$$

and the contextualized representation is

$$Y = AV.$$

These representations pass through multiple Transformer blocks. At the final layer, the hidden representation associated with the current prediction position is projected into vocabulary logits,

$$z = W_{\text{vocab}}h + b,$$

with

$$z \in \mathbb{R}^{|\mathcal{V}|}.$$

A softmax converts the logits into a probability distribution,

$$P(t_{n+1} = j) = \frac{e^{z_j}}{\sum_{k=1}^{|\mathcal{V}|} e^{z_k}}.$$

A decoding rule then selects or samples the next token.

Once the new token t_{n+1} has been generated, it becomes part of the context,

$$[t_1, \dots, t_n] \rightarrow [t_1, \dots, t_n, t_{n+1}],$$

and the process repeats.

2.13 Prefill and Decode

Autoregressive inference is commonly divided into two computational phases: *prefill* and *decode*.

During prefill, the entire initial prompt is processed simultaneously,

$$[t_1, t_2, \dots, t_T] \rightarrow K_{\text{cache}}, V_{\text{cache}}.$$

Because many prompt tokens can be processed in parallel, prefill can make effective use of large matrix operations.

After the prompt has been processed, the model enters the decode phase. New tokens are generated sequentially,

$$t_{T+1}, t_{T+2}, t_{T+3}, \dots$$

At each decoding step, the model computes the representations associated with the newly generated token and reuses the previously stored keys and values.

Prefill and decode therefore have different computational characteristics. Prefill typically involves large parallel matrix operations, whereas decoding consists of many small sequential operations and is often strongly constrained by memory bandwidth and KV-cache access.

2.14 The Transformer Block

An attention layer is only one component of a complete Transformer block. A common modern architecture uses normalization, attention, residual connections, and a position-wise feed-forward network.

In a pre-normalization Transformer, the first part of a block can be written approximately as

$$X' = X + \text{Attention}(\text{Norm}(X)).$$

The second part is

$$X'' = X' + \text{MLP}(\text{Norm}(X')).$$

The residual connections allow information to propagate through deep networks and provide short gradient paths during training.

The feed-forward network is applied independently to each token representation. A basic form is

$$\text{MLP}(x) = W_2\phi(W_1x + b_1) + b_2,$$

where ϕ is a nonlinear activation function.

Common activation choices include GELU, while many modern architectures use gated feed-forward networks such as SwiGLU.

The attention component mixes information across token positions, whereas the feed-forward component performs nonlinear processing independently at each token location. Repeated application of these two operations allows Transformers to construct increasingly complex contextual representations.

2.15 Tensor Dimensions in Attention

The dimensional structure of attention is useful for understanding both implementation and computational cost.

For a single attention head,

$$X \in \mathbb{R}^{n \times d_{\text{model}}},$$

$$Q, K \in \mathbb{R}^{n \times d_k},$$

and

$$V \in \mathbb{R}^{n \times d_v}.$$

Therefore,

$$QK^T : (n \times d_k)(d_k \times n) \rightarrow (n \times n).$$

The resulting matrix contains pairwise compatibility scores between sequence positions. After softmax,

$$A \in \mathbb{R}^{n \times n}.$$

Multiplication by the value matrix gives

$$AV : (n \times n)(n \times d_v) \rightarrow (n \times d_v).$$

Thus, the attention mechanism can be interpreted as consisting of two primary stages:

$$QK^T$$

constructs pairwise interaction scores, while

$$AV$$

performs the corresponding weighted aggregation of information.

2.16 Summary

A Transformer converts discrete input tokens into continuous representations, augments those representations with positional information, and repeatedly applies attention and position-wise nonlinear transformations.

The central attention computation is

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

For causal autoregressive models, this becomes

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V.$$

The matrices

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

determine which information is requested, which tokens are compatible with a request, and which information is ultimately aggregated.

Self-attention applies this mechanism within a single sequence, while cross-attention allows one representation to retrieve information from another. Positional mechanisms restore information about ordering, multi-head attention provides multiple learned interaction spaces, and residual feed-forward blocks repeatedly transform the resulting contextual representations.

During autoregressive inference, causal masking ensures that future information is inaccessible, while KV caching prevents unnecessary recomputation of previous key and value representations. Together, these components form the basis of modern decoder-only language models and many multimodal Transformer architectures.

3 Vision-Language-Action Models and Embodied AI

A **Vision-Language-Action (VLA)** model extends a vision-language model from perception and reasoning into control:

$$\boxed{\text{VLM: perception} + \text{language}}$$

$$\boxed{\text{VLA: perception} + \text{language} + \text{action}}$$

Instead of only describing the scene, a VLA predicts robot actions:

$$(o_t, \ell) \rightarrow \pi_\theta \rightarrow a_t,$$

where

- o_t is the current observation,
- ℓ is the language instruction,
- a_t is the robot action,
- π_θ is the learned policy.

A robot observation may contain multiple modalities:

$$o_t = (I_t, D_t, q_t, \dot{q}_t, f_t),$$

where I_t is RGB imagery, D_t is depth, q_t contains joint positions, $\dot{q}_t$ contains joint velocities, and f_t contains force/torque information.

3.1 VLA Architecture

A typical VLA architecture separately encodes different modalities:

$$I_t \rightarrow \text{Vision Encoder} \rightarrow z_V,$$

$$\ell \rightarrow \text{Language Encoder} \rightarrow z_L,$$

$$s_t \rightarrow \text{State Encoder} \rightarrow z_S.$$

These representations are fused by a multimodal model:

$$[z_V, z_L, z_S] \rightarrow \text{Multimodal Transformer} \rightarrow \text{Action Head}.$$

Thus,

$$a_t = f_\theta(z_V, z_L, z_S).$$

A useful mental model is

$$\boxed{\text{Observation} \rightarrow \text{Tokens} \rightarrow \text{Multimodal Transformer} \rightarrow \text{Action}}$$

3.2 Action Representations

There is no universal robot action representation. The action depends on the control interface.

Joint Position

$$a_t = [q_1^*, q_2^*, \dots, q_n^*].$$

Joint Velocity

$$a_t = [\dot{q}_1, \dots, \dot{q}_n].$$

Joint Torque

$$a_t = [\tau_1, \dots, \tau_n].$$

End-Effector Delta

$$a_t = [\Delta x, \Delta y, \Delta z, \Delta r_x, \Delta r_y, \Delta r_z, g],$$

where g represents a gripper command.

This can be written compactly as

$$a_t = [\Delta p_t, \Delta R_t, g_t].$$

Absolute Cartesian Pose

$$a_t = [x, y, z, q_w, q_x, q_y, q_z, g].$$

End-effector deltas are often useful because they provide a relatively robot-independent action representation.

3.3 Observation Tokenization

Transformers operate on sequences of vectors, so robot observations must first be converted into tokens.

For vision,

$$I_t \rightarrow \text{image patches} \rightarrow x_1, \dots, x_N.$$

Robot state can be projected using an MLP:

$$s_t = [q_t, \dot{q}_t, g_t],$$

$$x_s = W_s s_t + b_s.$$

Language is tokenized in the usual way.

The resulting multimodal sequence might be

$$X = [x_1^{\text{text}}, \dots, x_M^{\text{text}}, x_1^{\text{vision}}, \dots, x_N^{\text{vision}}, x_s].$$

A Transformer then models interactions across modalities:

$$H = \text{Transformer}(X).$$

3.4 Temporal Context

A single observation is often insufficient for robotics.

For example, one image may not reveal whether an object is moving left or right.

Therefore, policies may use observation histories:

$$o_{t-k:t} = [o_{t-k}, \dots, o_t].$$

Each observation can be encoded:

$$x_t = f(o_t),$$

and the temporal sequence becomes

$$[x_{t-k}, \dots, x_t].$$

This gives the model information about motion and temporal evolution.

3.5 Action Tokenization

One approach is to represent continuous actions as discrete tokens.

Suppose

$$\Delta x \in [-0.05, 0.05] \text{ m.}$$

The interval can be discretized into B bins:

$$[-0.05, 0.05] \rightarrow \{0, 1, \dots, B-1\}.$$

Action prediction can then be formulated similarly to next-token prediction:

$$P(a_t \mid o_{\leq t}, \ell).$$

This allows standard autoregressive Transformer machinery and cross-entropy training:

$$\mathcal{L} = -\log P(a_t^{\text{true}}).$$

The conceptual sequence becomes

$$\boxed{\text{Text Tokens} + \text{Vision Tokens} + \text{Action Tokens}}$$

processed by a common sequence model.

3.6 Discretization Error

Action tokenization introduces finite resolution.

For

$$a \in [a_{\min}, a_{\max}]$$

with B bins, the approximate action resolution is

$$\Delta a \approx \frac{a_{\max} - a_{\min}}{B}.$$

Therefore, discrete action prediction trades continuous precision for compatibility with token-based sequence modeling.

3.7 Continuous Action Prediction

Instead of discretizing actions, the network can directly predict real-valued controls:

$$a_t \in \mathbb{R}^{d_a}.$$

A simple action head is

$$\hat{a}_t = Wh_t + b.$$

Training can use regression:

$$\mathcal{L} = \|a_t - \hat{a}_t\|_2^2.$$

Huber loss or other regression losses may also be used.

The major advantage is that there is no quantization error.

3.8 Multimodal Action Distributions

Simple deterministic regression can fail when several actions are valid.

Suppose two possible actions are

$$a_1 = [-1, 0], \quad a_2 = [1, 0].$$

Minimizing mean-squared error may produce their average:

$$\hat{a} = [0, 0],$$

even though this may be a poor action.

The problem is that

$$p(a \mid o)$$

can be multimodal.

Instead of predicting only

$$\hat{a} = f(o),$$

we can model a distribution:

$$p(a \mid o),$$

and sample

$$a \sim p(a \mid o).$$

For example,

$$p(a \mid o) = 0.5p_{\text{left}} + 0.5p_{\text{right}}.$$

This motivates distributional action models such as mixture models, diffusion policies, flow matching, and autoregressive action prediction.

3.9 Action Chunking

A single-step policy predicts

$$a_t,$$

then later

$$a_{t+1},$$

and so forth.

Action chunking instead predicts multiple future actions simultaneously:

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H-1}]$$

where H is the chunk horizon.

If the action dimension is d_a ,

$$A_t \in \mathbb{R}^{H \times d_a}.$$

For example,

$$H = 16, \quad d_a = 7$$

requires predicting

$$16 \times 7 = 112$$

continuous values.

Action chunking provides three important benefits:

- improved temporal coherence,
- reduced model inference frequency,
- representation of short-horizon trajectory intent.

3.10 Receding-Horizon Execution

Predicting an entire chunk does not require executing the whole chunk open-loop.

A useful strategy is:

1. predict H future actions,
2. execute only the first few,
3. observe the environment again,
4. predict a new chunk.

Thus,

Predict Long	Execute Short
--------------	---------------

This combines trajectory coherence with closed-loop feedback.

3.11 Temporal Ensembling

When overlapping action chunks are predicted, several predictions may exist for the same future timestep.

For example,

$$A_t = [a_t, a_{t+1}, a_{t+2}, \dots]$$

and

$$A_{t+1} = [a'_{t+1}, a'_{t+2}, \dots].$$

Multiple predictions can be combined:

$$\hat{a}_{t+1} = \sum_i w_i a_{t+1}^{(i)}.$$

This can smooth the executed control trajectory.

3.12 Hierarchical Policies

Complex tasks naturally operate at multiple levels of abstraction.

For an instruction such as

Make me coffee.

a high-level policy may produce subgoals such as

1. find mug,
2. grasp mug,
3. move mug to machine,
4. operate machine,
5. retrieve mug.

The high-level policy can be represented as

$$g_t = \pi_H(o_t, \ell),$$

where g_t is a goal or subtask.

A low-level policy then produces motor actions:

$$a_t = \pi_L(o_t, g_t).$$

Therefore,

$\pi_H : \text{observation} \rightarrow \text{subgoal}$

and

$\pi_L : \text{observation} + \text{subgoal} \rightarrow \text{motor action}$

The high-level output might be a language instruction, skill ID, waypoint, target pose, or object-centric goal.

3.13 Why Hierarchy Matters

Robotic reasoning and control operate at very different timescales.

Semantic planning may operate near

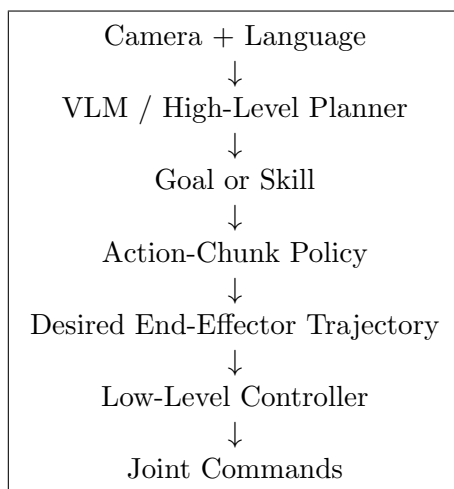
1 Hz,

while learned manipulation policies may operate at roughly

10–50 Hz,

and low-level motor controllers may run at hundreds or thousands of Hertz.

A conceptual robot stack is therefore



This avoids running expensive semantic reasoning at servo-control frequencies.

3.14 Action Chunking vs. Hierarchy

These concepts solve different problems.

Action chunking = temporal grouping of low-level actions

while

Hierarchy = abstraction across levels of control

They can be combined:

High-Level Policy $\rightarrow g_t \rightarrow$ Low-Level Policy $\rightarrow [a_t, \dots, a_{t+H-1}]$.

3.15 Behavior Cloning

Robot demonstrations can be represented as trajectories:

$$\tau = (o_1, a_1, o_2, a_2, \dots, o_T, a_T).$$

With language instruction ℓ , a dataset is

$$\mathcal{D} = \{(\ell_i, \tau_i)\}_{i=1}^N.$$

For discrete actions, behavior cloning can minimize

$$\mathcal{L} = - \sum_t \log \pi_\theta(a_t \mid o_{\leq t}, \ell).$$

For continuous regression,

$$\mathcal{L} = \sum_t \|a_t - \hat{a}_t\|_2^2.$$

3.16 Covariate Shift

A major problem with behavior cloning is that training observations come from the expert distribution:

$$s \sim p_{\text{expert}}(s).$$

During deployment, however, a small policy error can move the robot into a new state s' that was not represented in the demonstrations.

The policy is then operating off-distribution, which can cause another error, followed by another. This produces compounding error:

$\text{Small Error} \rightarrow \text{New State} \rightarrow \text{Distribution Shift} \rightarrow \text{Larger Error}$

This phenomenon is commonly called **covariate shift** in imitation learning.

Strategies mentioned for addressing it include closed-loop replanning, DAgger, reinforcement-learning fine-tuning, and data augmentation.

3.17 Discrete vs. Continuous Actions

Discrete / Tokenized Actions Advantages:

- naturally integrate with autoregressive Transformers,
- use cross-entropy objectives,
- can represent multimodality through token probabilities.

Disadvantages:

- introduce quantization error,
- may require large action vocabularies,
- may require sequential decoding.

Continuous Actions Advantages:

- natural representation for robot control,
- no quantization error,
- entire action vectors can be predicted directly.

Disadvantages:

- deterministic regression can average multiple valid action modes.

Neither representation is universally superior.

3.18 Interview Summary

VLA: A policy model that conditions on visual observations, language, and robot state to predict actions.

Observation Tokenization: Converts images, state, proprioception, and other observations into vector representations suitable for a sequence model.

Action Tokenization: Discretizes continuous robot actions so control prediction can be treated similarly to next-token prediction.

Continuous Actions: Predicts real-valued robot commands directly rather than discretizing them.

Action Chunking: Predicts several future actions in a single model inference.

Receding-Horizon Control: Predicts a long action chunk but executes only part of it before observing and replanning.

Temporal Ensembling: Combines overlapping predictions for the same future action to produce smoother control.

Hierarchical Policy: Separates slow high-level task reasoning from fast low-level motor execution.

Behavior Cloning: Learns a policy by matching demonstrated expert actions.

Covariate Shift: Deployment errors push the robot into states not represented in expert demonstrations, causing errors to compound.

3.19 Key Mental Model

The complete embodied-AI pipeline can be summarized as

Observation → Tokens → Multimodal Transformer → Intent → Action Chunk → Low-Level Controller
--

The central design questions are

How should observations be represented?
 How should actions be represented?
 Discrete or continuous actions?
 Single-step or chunked prediction?
 Flat or hierarchical policy?
 How frequently should the system replan?

For interview preparation, the most important relationships to remember are

$$(o_t, \ell) \rightarrow \pi_\theta \rightarrow a_t$$

$$A_t = [a_t, \dots, a_{t+H-1}]$$

and

$$g_t = \pi_H(o_t, \ell), \quad a_t = \pi_L(o_t, g_t).$$

4 Diffusion Models, Flow Matching, and Generative Robot Policies

Many robot-learning problems are naturally *multimodal*. Given the same observation and task, there may be several distinct action sequences that would all lead to success. A robot navigating around an obstacle, for example, may pass on either the left or the right. Similarly, a manipulator may approach an object from several valid directions.

This creates a fundamental difficulty for deterministic regression.

Suppose two valid action trajectories are

$$A_{\text{left}}$$

and

$$A_{\text{right}}.$$

A regression model trained with mean-squared error tends to predict the conditional mean,

$$\hat{A} \approx \frac{A_{\text{left}} + A_{\text{right}}}{2}.$$

However, the average of two valid trajectories is not necessarily itself valid. If the two trajectories correspond to passing an obstacle on opposite sides, their average may pass directly through the obstacle.

This motivates a different modeling objective. Rather than predicting a single action trajectory, the model should represent a conditional distribution

$$p(A|c),$$

where A denotes an action trajectory and c denotes the conditioning information available to the policy, such as images, language instructions, proprioceptive state, or task goals.

Two important families of generative models for this purpose are *diffusion models* and *flow-matching models*. Both transform a simple random distribution into a complex data distribution, but they formulate the transformation differently.

Diffusion models learn how to reverse a gradual corruption process, while flow-matching models learn a continuous velocity field that transports probability mass from a simple base distribution toward the data distribution.

4.1 Generative Modeling of Robot Actions

For robot control, the object being generated is often not a single action but an *action chunk*

$$A = [a_t, a_{t+1}, \dots, a_{t+H-1}],$$

where H is the prediction horizon.

If the condition is

$$c = (I_t, q_t, \ell),$$

then the policy may model

$$p(A|I_t, q_t, \ell),$$

where I_t is the current visual observation, q_t is the robot state, and ℓ is a language instruction.

This formulation allows the same observation to produce several possible trajectories, each corresponding to a valid mode of the action distribution.

Sampling a trajectory then becomes part of policy inference.

4.2 Diffusion Models

A diffusion model defines two complementary stochastic processes.

The *forward diffusion process* gradually corrupts a data sample by adding Gaussian noise.

The *reverse diffusion process* attempts to invert this corruption and transform noise back into a sample from the data distribution.

For robot policies, the clean sample may be an action trajectory

$$A_0 = [a_t, a_{t+1}, \dots, a_{t+H-1}].$$

The goal is to learn a model capable of generating samples approximately distributed according to

$$p(A_0|c).$$

The important idea is that training is performed on partially corrupted versions of real trajectories, while generation begins with noise.

4.3 The Forward Diffusion Process

Let

$$x_0 \sim p_{\text{data}}$$

denote a clean data sample.

A standard Denoising Diffusion Probabilistic Model (DDPM) defines a Markovian forward process

$$q(x_t|x_{t-1}) = \mathcal{N}\left(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right),$$

where β_t is a small positive noise parameter.

At every diffusion step, the sample is slightly attenuated and additional Gaussian noise is introduced.

It is convenient to define

$$\alpha_t = 1 - \beta_t$$

and the cumulative product

$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s.$$

A particularly important result is that x_t can be sampled directly from x_0 without explicitly simulating all preceding diffusion steps:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

where

$$\epsilon \sim \mathcal{N}(0, I).$$

This identity is central to practical diffusion training.

At small t , the coefficient

$$\sqrt{\bar{\alpha}_t}$$

is relatively large, so the sample still resembles the original data.

As t increases, the signal component decreases while the noise component increases.

At the final diffusion step,

$$x_T \approx \mathcal{N}(0, I).$$

Thus, the forward process gradually converts structured data into approximately Gaussian noise:

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_T.$$

The forward process is normally fixed mathematically rather than learned.

4.4 The Reverse Diffusion Process

Generation proceeds in the opposite direction.

One begins with

$$x_T \sim \mathcal{N}(0, I)$$

and attempts to construct

$$x_{T-1}, x_{T-2}, \dots, x_0.$$

The ideal reverse conditional distribution

$$q(x_{t-1}|x_t)$$

depends on the unknown data distribution and cannot generally be evaluated directly. A parameterized neural network is therefore used to approximate the reverse process:

$$p_\theta(x_{t-1}|x_t).$$

A common parameterization is

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t).$$

The network receives both the noisy sample x_t and the diffusion timestep t , because the correct denoising behavior depends strongly on how noisy the sample currently is.

Conceptually, the network must answer:

Given this corrupted sample and its noise level, how should it be transformed toward a cleaner sample?

4.5 Noise Prediction

A particularly common parameterization does not predict the clean sample or the reverse mean directly. Instead, it predicts the Gaussian noise that was used to construct x_t .

From the forward relation,

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon,$$

a neural network predicts

$$\hat{\epsilon} = \epsilon_\theta(x_t, t).$$

The canonical training objective is

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{x_0, t, \epsilon} \left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2 \right].$$

Training therefore consists of repeatedly performing the following procedure:

1. sample a clean data point x_0 ,
2. sample a diffusion timestep t ,
3. sample Gaussian noise ϵ ,
4. construct x_t ,
5. ask the model to predict ϵ ,
6. minimize the noise-prediction error.

One appealing aspect of this formulation is that the objective is simply a regression loss even though the resulting model represents a complicated probability distribution.

4.6 Why Predicting Noise Is Sufficient

The connection between noise prediction and denoising follows directly from the forward equation. Starting from

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon,$$

we can rearrange to obtain

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}.$$

Therefore, if the model accurately estimates ϵ , it can also infer an estimate of the clean sample x_0 .

The denoising network can consequently be parameterized in several equivalent or closely related ways.

It may predict the added noise,

$$\epsilon_\theta(x_t, t) \approx \epsilon,$$

the clean sample,

$$x_\theta(x_t, t) \approx x_0,$$

or a derived quantity such as a velocity parameterization,

$$v = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1 - \bar{\alpha}_t}x_0.$$

The phrase *diffusion objective* therefore does not uniquely imply noise prediction. Noise prediction is simply one particularly common formulation.

4.7 Conditional Diffusion

For robot learning, unconditional generation is rarely sufficient.

The model should generate actions appropriate for the current scene and task. Instead of learning

$$p(A),$$

we therefore want

$$p(A|c).$$

The conditioning variable may contain

$$c = (I_t, q_t, \ell),$$

where I_t represents visual input, q_t represents proprioceptive state, and ℓ represents a language instruction.

The denoiser becomes

$$\epsilon_\theta(x_t, t, c).$$

For an action chunk

$$A_0 = [a_t, \dots, a_{t+H-1}],$$

training constructs

$$A_\tau = \sqrt{\bar{\alpha}_\tau} A_0 + \sqrt{1 - \bar{\alpha}_\tau} \epsilon,$$

and predicts

$$\hat{\epsilon} = \epsilon_\theta(A_\tau, \tau, c).$$

The training loss remains

$$\mathcal{L} = \|\epsilon - \hat{\epsilon}\|_2^2.$$

During inference, the model begins with

$$A_T \sim \mathcal{N}(0, I)$$

and repeatedly denoises while conditioning on c .

The final output is a trajectory consistent with the current observation and task.

This is the basic idea behind a diffusion policy.

4.8 Why Diffusion Is Attractive for Robot Policies

Diffusion is particularly useful when the conditional action distribution is multimodal.

Suppose the robot may validly execute either

$$A_{\text{left}}$$

or

$$A_{\text{right}}.$$

A deterministic regressor may predict an undesirable average between the two.

A diffusion model instead represents

$$p(A|c)$$

and can sample

$$A \sim p(A|c).$$

Different random initial conditions can therefore lead to different valid modes.

One sample might produce the left-side strategy, while another might produce the right-side strategy.

The model does not need to collapse both strategies into a single conditional mean.

4.9 How Conditioning Enters the Network

There are several ways for external context to influence a generative model.

The simplest is concatenation:

$$[x_t; c].$$

The network receives the noisy sample and conditioning variables as a joint input.

Another approach is feature-wise modulation. The condition generates scale and shift parameters,

$$h' = \gamma(c)h + \beta(c).$$

This allows the condition to modulate intermediate network features.

For Transformer-based models, cross-attention is especially natural.

Action representations may produce queries,

$$Q = X_{\text{action}} W_Q,$$

while visual or language representations produce keys and values:

$$K = X_{\text{obs}} W_K, \quad V = X_{\text{obs}} W_V.$$

Cross-attention then computes

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V.$$

The generated trajectory can therefore selectively attend to the portions of the scene or instruction that are relevant to each action.

4.10 DDPM Sampling

The DDPM formulation uses a stochastic reverse process.

A conceptual reverse step is

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z,$$

where

$$z \sim \mathcal{N}(0, I).$$

Thus, generation proceeds through a sequence of stochastic transitions:

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

Traditional DDPM formulations often use a large number of denoising steps.

Every step requires a neural-network evaluation, so generation can be computationally expensive.

This cost is especially important in robotics, where action-generation latency may directly limit control frequency.

4.11 DDIM Sampling

The Denoising Diffusion Implicit Model (DDIM) provides a different sampling formulation that can reuse the same underlying denoising network.

DDIM allows the reverse process to become deterministic by setting the stochastic component to zero:

$$\sigma_t = 0.$$

A reverse update then has the form

$$x_{t-1} = f(x_t, \epsilon_\theta(x_t, t)).$$

For a fixed initial noise sample and timestep schedule, the resulting trajectory can therefore be deterministic.

DDIM also makes it possible to use a reduced sequence of denoising steps.

The key distinction is not that DDPM and DDIM require completely different learned networks. Rather, DDIM primarily changes the sampling trajectory through the diffusion process.

Thus, at a high level,

DDPM

is associated with stochastic reverse sampling, while

DDIM

allows deterministic or less stochastic sampling and often supports fewer network evaluations.

4.12 Diffusion Transformers

The diffusion formulation specifies a generative objective and sampling process, but it does not dictate the architecture of the denoising network.

Historically, U-Net-style architectures have been common diffusion backbones.

A *Diffusion Transformer* (DiT) instead uses a Transformer as the network responsible for predicting the denoising quantity.

Conceptually,

$$x_t \rightarrow \text{tokenization} \rightarrow \text{Transformer} \rightarrow \hat{\epsilon}.$$

For robot action generation, suppose

$$A_\tau \in \mathbb{R}^{H \times d_a}.$$

Each action timestep can be embedded as a token:

$$a_i \rightarrow e_i.$$

The Transformer can then process the sequence of action tokens along with additional information such as

generative timestep,

trajectory position,

visual context,

robot state,

and

language instructions.

If E denotes the action-token sequence, a simplified model may be written as

$$H = \text{Transformer}(E, c, \tau),$$

followed by

$$\hat{e} = WH.$$

The important distinction is that DiT is an *architecture*, not a training objective.

A Transformer can be used inside a diffusion model, a flow-matching model, or another generative framework.

4.13 Why Transformers Are Natural for Action Trajectories

Robot action chunks are inherently sequential:

$$a_t, a_{t+1}, \dots, a_{t+H-1}.$$

Actions at different positions in the sequence are correlated.

For example, an early reaching motion affects the appropriate orientation and gripper behavior later in the trajectory.

Self-attention gives the model a direct mechanism for representing such dependencies.

Cross-attention also provides a convenient way to condition action generation on visual or language tokens.

For this reason, Transformer-based generative models fit naturally with trajectory prediction.

4.14 Flow Matching

Flow matching provides another approach to generative modeling.

Rather than defining a stochastic noising process and learning how to reverse it, flow matching directly learns a *velocity field* that transports samples from a simple base distribution to the data distribution.

Let

$$x_0 \sim p_0$$

denote a sample from a simple distribution, typically Gaussian noise, and let

$$x_1 \sim p_{\text{data}}$$

denote a sample from the data distribution.
 We define a continuous path

$$x_\tau, \quad \tau \in [0, 1],$$

that connects the two.

The simplest example is linear interpolation:

$$x_\tau = (1 - \tau)x_0 + \tau x_1.$$

At

$$\tau = 0,$$

we have

$$x_\tau = x_0,$$

while at

$$\tau = 1,$$

we have

$$x_\tau = x_1.$$

4.15 The Velocity Field

Differentiating the linear path gives

$$\frac{dx_\tau}{d\tau} = x_1 - x_0.$$

The desired velocity is therefore

$$u_\tau = x_1 - x_0$$

for this simple interpolation.

A neural network

$$v_\theta(x_\tau, \tau)$$

is trained to predict the target velocity.

A simple flow-matching objective is

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \left[\|v_\theta(x_\tau, \tau) - u_\tau\|_2^2 \right].$$

The model is therefore not directly predicting noise or a clean sample. It predicts how the current sample should move through the generative state space.

4.16 Generation with Flow Matching

Once the velocity field has been learned, generation begins with

$$x(0) \sim p_0.$$

The generated sample evolves according to the ordinary differential equation

$$\frac{dx}{d\tau} = v_\theta(x, \tau).$$

Integrating from

$$\tau = 0$$

to

$$\tau = 1$$

transports a noise sample toward the learned data distribution.

The model can be interpreted as answering the following question:

Given the current sample and generative time, in what direction and at what speed should the sample move?

Generation therefore corresponds to following a learned vector field.

4.17 Conditional Flow Matching for Robot Control

As with diffusion, robot policies require conditional generation.

Let

$$A = [a_t, \dots, a_{t+H-1}]$$

denote an action chunk, and let

$$c = (I_t, q_t, \ell)$$

denote the current observation and task condition.

Generation begins with a random action trajectory

$$A(0) \sim \mathcal{N}(0, I).$$

The model defines a conditional velocity field

$$v_\theta(A, \tau, c).$$

Inference then solves

$$\frac{dA}{d\tau} = v_\theta(A, \tau, c).$$

At

$$\tau = 1,$$

the trajectory

$$A(1)$$

should correspond to a plausible action sequence for the given condition.

Different initial noise samples can produce different trajectories, allowing the model to represent multimodal behavior.

4.18 Diffusion and Flow Matching

Diffusion and flow matching are conceptually different but mathematically related.

Diffusion is often introduced through a stochastic differential equation,

$$dx = f(x, t) dt + g(t) dW_t,$$

where dW_t represents Brownian noise.

The forward process gradually destroys structure, and the learned reverse dynamics reconstruct the data distribution.

Flow matching instead directly learns an ODE,

$$\frac{dx}{dt} = v_\theta(x, t).$$

At a conceptual level,

diffusion

asks how to remove corruption, whereas

flow matching

asks how samples should move continuously through probability space.

The distinction should not be overstated. Diffusion models can be associated with probability-flow ODEs that reproduce the same marginal distributions as the corresponding stochastic process.

A more precise description is therefore:

Diffusion typically begins from a stochastic corruption process and learns reverse dynamics, whereas flow matching directly trains a vector field that transports probability mass between distributions.

4.19 Generation Cost and Control Latency

Generative policies are computationally more expensive than direct regression because they typically require several neural-network evaluations for every generated action sequence.

Suppose a robot replans at

20 Hz.

The available time for generating the next action chunk is then roughly

50 ms

before accounting for perception, communication, and low-level control overhead.

If trajectory generation requires hundreds of network evaluations, real-time deployment may be impractical.

Sampling efficiency is therefore not merely a computational concern in robotics. It directly affects achievable control frequency.

This is one reason reduced-step diffusion samplers and flow-based methods are attractive for embodied systems.

4.20 Generative Time and Physical Time

Robot generative policies involve two distinct time coordinates that should not be confused.

Physical control time is indexed by t . An action trajectory is

$$A = [a_t, a_{t+1}, \dots, a_{t+H-1}].$$

These indices correspond to actions that will be executed at successive robot-control timesteps.

The generative process introduces a second time variable, which is useful to denote by τ .

For example,

$$A_{\tau=0}$$

may correspond to a random trajectory, while

$$A_{\tau=1}$$

corresponds to the generated clean action trajectory.

Thus,

$$t = \text{physical robot time,}$$

while

$$\tau = \text{generative time.}$$

A diffusion or flow model therefore transforms an entire trajectory through generative time while the positions within that trajectory refer to physical time.

This distinction is especially important when reading robot diffusion-policy and flow-matching equations.

4.21 Direct Regression and Generative Policies

It is useful to compare generative action models with direct regression.

A deterministic regressor computes

$$\hat{A} = f_{\theta}(c).$$

Generation requires only one forward pass, making this approach computationally attractive.

However, if the conditional distribution

$$p(A|c)$$

has several modes, deterministic regression tends to produce a central tendency such as the mean.

Diffusion and flow-matching models instead introduce a random variable at inference time. For diffusion,

$$A_T \sim \mathcal{N}(0, I),$$

and repeated denoising transforms this noise into a trajectory. For flow matching,

$$A(0) \sim \mathcal{N}(0, I),$$

and ODE integration transports the random trajectory toward the data distribution. Different random initializations may therefore produce different valid behaviors.

4.22 A Robot Manipulation Example

Suppose a robot receives the condition

$$c = (I_t, q_t, \text{“pick up the cup”})$$

and must generate a 32-step action trajectory with seven action dimensions per timestep:

$$A \in \mathbb{R}^{32 \times 7}.$$

The seven action components might represent six-dimensional end-effector motion together with a gripper command.

In a diffusion policy, inference begins with

$$A_T \sim \mathcal{N}(0, I).$$

At every generative step, the model evaluates

$$\hat{e} = \epsilon_\theta(A_\tau, \tau, c)$$

and updates the current trajectory toward a less noisy version. After repeated updates,

$$A_T \rightarrow \cdots \rightarrow A_0,$$

where A_0 is a coherent action sequence.

In a flow-matching policy, inference instead begins with

$$A(0) \sim \mathcal{N}(0, I)$$

and integrates

$$\frac{dA}{d\tau} = v_\theta(A, \tau, c)$$

until

$$\tau = 1.$$

The result

$$A(1)$$

is the generated trajectory.

Both models can therefore map the same observation into different plausible action sequences depending on the random initial condition.

4.23 Choosing a Generative Formulation

The different approaches can be understood in terms of what the network predicts and how generation proceeds.

A direct regression model predicts the action trajectory itself:

$$\hat{A} = f_{\theta}(c).$$

A DDPM-style diffusion model usually predicts a denoising quantity such as

$$\epsilon, \quad x_0,$$

or a related parameterization and generates samples through iterative denoising.

A DDIM sampler uses the same general denoiser but follows a different reverse trajectory that may be deterministic and use fewer steps.

A flow-matching model predicts

$$v_{\theta}(x, \tau),$$

a time-dependent velocity field, and generates samples by integrating an ODE.

A DiT describes the architecture used to implement the neural network rather than the generative objective itself.

It is therefore possible to combine different design choices. For example,

Transformer architecture + diffusion objective

and

Transformer architecture + flow-matching objective

are both valid model designs.

4.24 The Broader Role of Generative Policies

Generative robot policies are useful because robot behavior is rarely described by a unique correct action.

A task may permit several grasp poses, several collision-free trajectories, or several different high-level strategies.

A useful policy should therefore often represent

$$p(A|c)$$

rather than only its conditional mean.

This perspective unifies several modern action-generation methods.

Diffusion models construct the action distribution through iterative denoising.

Flow-matching models construct it through continuous probability transport.

Autoregressive models factor the action distribution into a sequence of conditional predictions.

Mixture models explicitly represent several modes.

Although these approaches differ mathematically, they address the same fundamental issue: uncertainty and multimodality in future robot behavior.

4.25 Summary

Generative robot policies model distributions over actions or trajectories rather than predicting a single deterministic control vector. This is useful because many robot tasks admit several different successful behaviors, and averaging those behaviors can produce an invalid solution.

Diffusion models introduce a forward process that gradually converts data into Gaussian noise,

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon,$$

and learn a reverse process that reconstructs data from noisy samples.

A common diffusion objective predicts the added noise:

$$\mathcal{L}_{\text{diff}} = \mathbb{E} \left[\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2 \right].$$

For robotics, conditioning allows the generated trajectory to depend on current observations, language, and robot state.

DDPM and DDIM primarily differ in the reverse sampling procedure. DDPM uses stochastic reverse sampling, whereas DDIM permits deterministic or reduced-step trajectories using the same general denoising model.

A Diffusion Transformer uses a Transformer as the denoising backbone. DiT is therefore an architectural choice rather than a distinct generative objective.

Flow matching takes a different perspective. It learns a vector field

$$v_\theta(x, \tau)$$

such that generation can be performed by integrating

$$\frac{dx}{d\tau} = v_\theta(x, \tau).$$

Conditional flow matching applies the same idea to robot actions:

$$\frac{dA}{d\tau} = v_\theta(A, \tau, c).$$

Diffusion and flow matching are mathematically related approaches to transporting a simple base distribution into a complex data distribution.

For robot control, the central practical considerations are multimodality, conditioning, trajectory coherence, and inference latency. A generative policy is valuable when multiple action sequences may be correct, but its sampling procedure must be efficient enough to satisfy the robot’s control-rate requirements.

The central distinction can therefore be summarized as

Diffusion: learn to reverse corruption

and

Flow matching: learn the velocity of probability transport.

Both ultimately seek to model

$$p(A|c),$$

the distribution of valid action trajectories conditioned on the robot’s current context.

5 Behavior Cloning, DAgger, and Trajectory Modeling

Imitation learning attempts to construct a policy by learning from demonstrations of desired behavior. In robotics, this is attractive because specifying a reward function for a complicated manipulation or navigation task can be difficult, whereas it may be comparatively easy for a human operator or an existing controller to demonstrate the desired behavior.

The simplest form of imitation learning is *behavior cloning*. Behavior cloning treats policy learning as a supervised learning problem: observations are used as inputs and demonstrated actions are used as targets.

This formulation is conceptually simple, but sequential decision-making introduces a difficulty that does not normally appear in static supervised learning. A prediction made by a control policy changes the physical system and therefore changes the inputs that the policy will receive in the future. Even a small error can move the robot into states that were rarely represented in the expert demonstrations. Subsequent predictions may then become less reliable, causing errors to accumulate over time.

This phenomenon connects several important concepts:

behavior cloning $\rightarrow$ covariate shift $\rightarrow$ compounding error.

The DAgger algorithm addresses this problem by collecting training examples from states visited by the learned policy itself rather than only from states visited by the expert.

A separate but complementary issue concerns the temporal representation of actions. A policy can predict one action at a time, or it can predict an entire short trajectory. The latter approach, known as *action chunking*, allows the model to capture temporal structure and multiple possible short-horizon behaviors.

These ideas can be organized into two related themes:

behavior cloning $\rightarrow$ distribution shift $\rightarrow$ DAgger,

and

trajectory distributions $\rightarrow$ action chunking $\rightarrow$ short-horizon trajectory modeling.

5.1 Behavior Cloning

Suppose an expert generates a trajectory

$$\tau = (o_1, a_1, o_2, a_2, \dots, o_T, a_T),$$

where o_t denotes the observation available to the policy and a_t denotes the expert action. From one or more trajectories, we construct a dataset of observation–action pairs,

$$\mathcal{D} = \{(o_i, a_i)\}_{i=1}^N.$$

A parameterized policy

$$\pi_\theta(a|o)$$

is trained to reproduce the demonstrated action.

For a deterministic continuous-action policy, a common objective is mean-squared error:

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N} \sum_{i=1}^N \|a_i - \pi_\theta(o_i)\|_2^2.$$

For discrete or stochastic actions, the corresponding maximum-likelihood objective is

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \pi_\theta(a_i|o_i).$$

Behavior cloning is therefore ordinary supervised learning applied to expert demonstrations.

Its main attraction is simplicity. The learner does not need an explicitly designed reward function, online exploration, or reinforcement learning. If a sufficiently large collection of high-quality demonstrations is available, useful policies can often be learned directly from them.

The central limitation is that the supervised-learning objective is evaluated on observations from the demonstration dataset, while the learned policy is eventually evaluated on observations produced by its own actions.

5.2 Why Sequential Prediction Is Different from Static Prediction

Consider a conventional image-classification problem. If a classifier makes an incorrect prediction for one image, the mistake does not normally change the next image presented to the model.

Control is fundamentally different.

The action produced at time t changes the physical system,

$$s_t \rightarrow a_t \rightarrow s_{t+1},$$

and therefore changes the observation available to the policy at the next timestep.

Consequently,

the policy’s output at time t

influences

the policy’s input at time $t + 1$.

This feedback loop is the reason small prediction errors can have effects far beyond a single timestep.

Suppose an expert policy keeps a system near a nominal condition

$$x \approx 0.$$

The training dataset therefore contains many examples near this region.

Now suppose the learned policy makes a small error and moves the system to

$$x = 0.05.$$

If this state is uncommon in the demonstrations, the prediction made there may be less accurate. The next state might become

$$x = 0.10,$$

followed by

$$x = 0.20,$$

and so forth.

The important point is that errors are not independent. One error changes the distribution of future observations and can make later errors more likely.

5.3 Covariate Shift in Imitation Learning

Let π_E denote the expert policy.

When the expert generates demonstrations, it induces a distribution over states,

$$s \sim d_{\pi_E}(s).$$

Behavior cloning is trained primarily using samples from this distribution.

At deployment, however, the learned policy π_θ generates its own trajectories and therefore induces a different state distribution,

$$s \sim d_{\pi_\theta}(s).$$

In general,

$$d_{\pi_\theta}(s) \neq d_{\pi_E}(s).$$

This discrepancy is the central distribution-shift problem in behavior cloning.

It may also be written as

$$p_{\text{train}}(s) \neq p_{\text{deployment}}(s).$$

The important feature of this shift is that it is *policy-induced*. The learner is not merely tested on a randomly different dataset. Its own decisions determine which future states it will encounter.

This creates the feedback process

$$\text{prediction error} \rightarrow \text{new state} \rightarrow \text{less familiar observation} \rightarrow \text{larger prediction error}.$$

5.4 Compounding Error

This phenomenon is often described as *compounding error*.

Suppose a learned policy has a small probability ϵ of making a poor decision at each timestep. If errors were independent and did not affect future states, one might expect error to accumulate approximately linearly with the horizon.

In a sequential control problem, however, the first error changes the state distribution. The policy may then encounter observations for which its prediction error is larger than it was on the expert distribution.

Therefore, the consequences of errors can grow more rapidly than a simple sum of independent mistakes would suggest.

The essential distinction is

static supervised learning: prediction does not change the next input,
whereas

control: prediction changes the next input.

This is one of the fundamental reasons imitation learning is harder than ordinary supervised learning.

5.5 Dataset Aggregation

DAgger, short for *Dataset Aggregation*, was introduced to address the state-distribution mismatch created by behavior cloning.

The central idea is simple: instead of training only on states visited by the expert, the learner should also be trained on states that it visits itself.

Suppose the initial expert dataset is

$$\mathcal{D}_0.$$

A behavior-cloned policy

$$\pi_1$$

is trained from this dataset.

The policy is then deployed and generates states

$$s_1, s_2, \dots$$

according to its own state-visitation distribution.

For each visited state s_t , the expert is asked what action should have been taken:

$$a_t^* = \pi_E(s_t).$$

These newly labeled examples are added to the dataset:

$$\mathcal{D}_1 = \mathcal{D}_0 \cup \{(s_t, \pi_E(s_t))\}.$$

A new policy π_2 is then trained on the enlarged dataset.

The process is repeated:

$$\pi_k \rightarrow \text{run learner} \rightarrow \text{collect visited states} \rightarrow \text{obtain expert labels} \rightarrow \mathcal{D}_{k+1} \rightarrow \pi_{k+1}.$$

The important change is that the training distribution gradually incorporates states drawn from

$$d_{\pi_\theta},$$

which is the distribution the learner must actually handle at deployment.

5.6 Why DAgger Improves Robustness

Consider a lane-following system.

An expert may provide demonstrations in which the vehicle is almost always centered in the lane. The resulting dataset therefore contains many examples of nominal driving and very few examples of substantial lateral error.

A learned policy may eventually drift into states such as

20 cm right of center,

40 cm right of center,

or

rotated toward the road edge.

A behavior-cloned policy may have little experience with these states.

DAgger deliberately collects them and asks the expert to specify corrective behavior, for example

steer left,

steer left more aggressively,

or

reduce speed.

The resulting dataset contains not only nominal demonstrations but also recovery behavior.

This distinction is important. A robust controller must often know how to recover from deviations rather than merely imitate trajectories that never deviate.

5.7 Expert Labeling and Expert Control

The expert does not necessarily have to control the system continuously during DAgger data collection.

The learned policy can generate the trajectory,

$$s \sim d_{\pi_\theta},$$

while the expert provides only the target action

$$\pi_E(s)$$

for each encountered state.

This separation is useful because the goal of DAgger is specifically to obtain expert supervision on states distributed according to the learner.

In practice, however, safety may require the expert to retain some control. One possible approach is to use a mixture policy

$$\pi_{\text{mix}} = \beta \pi_E + (1 - \beta) \pi_\theta,$$

where β controls the contribution of expert behavior.

As training progresses and the learner becomes more reliable, β can be reduced.

5.8 Behavior Cloning and DAgger

Behavior cloning and DAgger solve related but distinct versions of the imitation-learning problem.

Behavior cloning trains on a fixed collection of expert demonstrations. It is simple, efficient, and often provides a strong initial policy.

DAgger introduces an iterative data-collection loop in which learner-visited states are labeled by the expert.

The advantage is that the dataset becomes better matched to the policy’s deployment distribution.

The disadvantage is additional operational cost. The expert must remain available during training, and repeatedly running the policy on a physical robot can be expensive.

The distinction can be summarized as

behavior cloning: learn from expert-visited states,

versus

DAgger: learn from learner-visited states labeled by the expert.

5.9 Trajectory Distributions

The distribution-shift problem becomes clearer when imitation learning is described in terms of complete trajectories.

A trajectory can be written as

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T).$$

Suppose the initial state follows

$$p(s_0),$$

the policy is

$$\pi(a_t|s_t),$$

and the environment dynamics are

$$p(s_{t+1}|s_t, a_t).$$

The policy and environment together induce the trajectory distribution

$$p_\pi(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t) p(s_{t+1}|s_t, a_t).$$

This equation reveals three sources of variation in a trajectory:

1. the initial state,
2. the actions selected by the policy,
3. the environment’s response to those actions.

Thus, changing the policy changes not merely the action distribution but the distribution of entire future rollouts.

5.10 Local Imitation Error Versus Rollout Quality

Behavior cloning optimizes a local objective such as

$$\pi_\theta(a|s) \approx \pi_E(a|s)$$

on states sampled from the expert dataset.

However, the quantity that ultimately matters in deployment is the quality of complete trajectories generated by the policy.

Ideally, one would like

$$p_{\pi_\theta}(\tau) \approx p_{\pi_E}(\tau).$$

These two requirements are not equivalent.

A policy may achieve very small behavior-cloning loss,

$$\mathcal{L}_{\text{BC}} \approx 0,$$

on expert states while still producing trajectories that differ substantially from expert behavior.

The reason is again feedback. A small action difference changes the next state, which changes the next policy input, and the trajectories gradually diverge.

5.11 State-Visitation Distributions

The trajectory distribution induces a state-visitation distribution

$$d_\pi(s).$$

Conceptually,

$$\text{policy} \rightarrow \text{trajectory distribution} \rightarrow \text{state-visitation distribution}.$$

This connects trajectory modeling directly to covariate shift.

The expert induces

$$p_{\pi_E}(\tau)$$

and therefore

$$d_{\pi_E}(s),$$

while the learned policy induces

$$p_{\pi_\theta}(\tau)$$

and therefore

$$d_{\pi_\theta}(s).$$

Behavior cloning becomes fragile when these state distributions diverge.

DAgger attempts to reduce the practical consequences of this divergence by adding expert supervision to states drawn from the learner's own distribution.

5.12 Multimodal Trajectory Distributions

A second difficulty in robot learning is that many tasks have more than one valid solution.

Consider a robot attempting to reach and grasp an object. It may approach the object from the left,

$$\tau_1 = \text{left-side approach,}$$

from the right,

$$\tau_2 = \text{right-side approach,}$$

or by first lifting above an obstacle,

$$\tau_3 = \text{overhead approach.}$$

All three trajectories may successfully accomplish the task.

The conditional trajectory distribution

$$p(\tau|o)$$

can therefore be multimodal.

This creates difficulty for simple deterministic regression.

Suppose two equally valid actions are

$$a_L = -1$$

and

$$a_R = +1.$$

If a deterministic policy is trained using mean-squared error, the optimal prediction under equal probability may be

$$\hat{a} = 0.$$

Yet 0 may correspond to a behavior that is invalid.

For instance, the two valid actions might represent moving around an obstacle from the left or right, while their average corresponds to driving directly into it.

Therefore,

$$\text{average of valid behaviors} \neq \text{necessarily a valid behavior.}$$

This problem motivates policies that represent full action distributions rather than only their conditional mean.

Examples include mixture models, autoregressive action models, diffusion policies, and flow-matching policies.

5.13 One-Step Action Prediction

A conventional policy predicts one action at each timestep:

$$a_t = \pi(o_t).$$

Probabilistically, it models a conditional distribution such as

$$p(a_t|o_t).$$

The policy is then queried again at the next timestep using the newly observed state.

This gives the controller frequent feedback and allows rapid reaction to environmental changes.

However, a one-step representation contains little explicit information about the temporal structure of future actions.

Many robot behaviors are naturally defined as coordinated sequences rather than isolated commands.

5.14 Action Chunking

Action chunking changes the policy output from one action to a sequence of future actions:

$$A_t = [a_t, a_{t+1}, \dots, a_{t+H-1}],$$

where H denotes the chunk horizon.

If an individual action has dimension d_a , then

$$A_t \in \mathbb{R}^{H \times d_a}.$$

For example, suppose a manipulator uses a seven-dimensional action,

$$d_a = 7,$$

consisting of six end-effector motion components and a gripper command.

If the model predicts

$$H = 16$$

future actions, then each inference produces

$$16 \times 7 = 112$$

continuous values.

The policy is no longer modeling only

$$p(a_t|o_t),$$

but a short-horizon distribution more like

$$p(a_{t:t+H-1}|o_t).$$

5.15 Temporal Structure in Action Chunks

Action chunking allows the policy to model correlations between future control commands.

Consider a grasping motion. A reasonable action sequence may be

approach $\rightarrow$ slow down $\rightarrow$ close gripper $\rightarrow$ lift.

These actions are not independent. The closing action should occur only after the hand reaches an appropriate pose, and lifting should occur only after the grasp has been established.

Predicting the sequence jointly allows the model to represent these dependencies directly.

A one-step policy, by contrast, repeatedly solves a local decision problem. Temporal consistency must emerge implicitly through the closed-loop interaction between successive predictions.

5.16 Action Chunking as Local Trajectory Modeling

An action chunk can be regarded as a short trajectory.

Instead of modeling

$$p(a_t|o_t),$$

the policy models

$$p(A_t|o_t).$$

This perspective becomes particularly important for generative action policies.

For the same observation o_t , a model may generate one valid chunk

$$A^{(1)} = \text{left-side grasp},$$

or another

$$A^{(2)} = \text{right-side grasp}.$$

Thus, action chunking creates a natural representation for multimodal short-horizon behavior.

Diffusion policies and flow-matching policies often operate on exactly this type of object: a distribution over complete action chunks.

5.17 Reducing Myopic Behavior

One-step policies can sometimes behave myopically because they optimize only the immediate next action.

Consider navigation around an obstacle.

The locally attractive action may point directly toward the goal. Yet successful motion may first require moving away from the goal:

move sideways $\rightarrow$ move forward.

A short action chunk can explicitly encode such temporal structure.

In this sense, action chunking introduces a limited form of trajectory-level reasoning even when the policy does not contain a full planning algorithm.

5.18 The Open-Loop Limitation

Longer action chunks also create a new problem.

Suppose the policy predicts

$$[a_t, \dots, a_{t+31}]$$

and executes all 32 actions without observing the environment again.

During this interval, execution is effectively open-loop.

If an object moves, a grasp slips, or the dynamics differ from expectation, later actions in the chunk may no longer be appropriate.

Increasing chunk length therefore introduces a tradeoff.

Longer chunks can improve temporal coherence and reduce the frequency of expensive model inference, but they also reduce the frequency of closed-loop feedback.

5.19 Receding-Horizon Execution

A common solution is to predict a relatively long chunk but execute only part of it.

Suppose the policy predicts H actions,

$$A_t = [a_t, \dots, a_{t+H-1}],$$

but executes only the first K , where

$$K < H.$$

The robot then acquires a new observation and predicts another chunk.

For example,

$$H = 32, \quad K = 4.$$

The procedure becomes

observe $\rightarrow$ predict 32 actions $\rightarrow$ execute 4 $\rightarrow$ observe again $\rightarrow$ replan.

This is similar in spirit to receding-horizon or model-predictive control: plan farther into the future than the controller actually commits to executing.

The result combines short-horizon trajectory coherence with feedback robustness.

5.20 Overlapping Chunks and Temporal Ensembling

If the model replans frequently, different action chunks overlap.

At time t , the model may predict

$$[a_t^{(t)}, a_{t+1}^{(t)}, a_{t+2}^{(t)}, \dots],$$

while at time $t + 1$ it predicts

$$[a_{t+1}^{(t+1)}, a_{t+2}^{(t+1)}, \dots].$$

There are now multiple predictions for the same future action.

These estimates can be combined. One weighted averaging rule is

$$\hat{a}_t = \frac{\sum_{k=0}^K w_k a_t^{(t-k)}}{\sum_{k=0}^K w_k}.$$

This procedure is commonly referred to as *temporal ensembling*.

The weights w_k can be chosen so that newer predictions receive greater importance than older predictions.

Averaging overlapping predictions can reduce high-frequency jitter and make execution smoother.

5.21 Action Chunking and Covariate Shift

Action chunking can improve temporal consistency, but it does not remove the distribution-shift problem of imitation learning.

A chunked policy still controls the system. Its actions change future states, and those states may drift outside the distribution represented in the demonstrations.

Thus, a chunked policy may still encounter observations satisfying

$$o \not\sim d_{\pi_E}.$$

Dagger and action chunking should therefore be understood as addressing different problems.

Dagger addresses *which states the policy is trained on*.

Action chunking addresses *how future actions are represented and predicted*.

They can be used together.

For example, a Dagger-style procedure could collect learner-visited observations while an action-chunk policy is trained to predict expert short-horizon trajectories from those observations.

5.22 A Unified View

Suppose an expert dataset contains trajectories

$$\mathcal{D}_E = \{\tau_1, \dots, \tau_N\}.$$

Behavior cloning trains a policy

$$\pi_\theta$$

using observation–action pairs extracted from these trajectories.

Because the learner is imperfect, deployment induces a state distribution

$$d_{\pi_\theta}$$

that may differ from

$$d_{\pi_E}.$$

This is the core covariate-shift problem.

Dagger addresses the mismatch by collecting states

$$s \sim d_{\pi_\theta}$$

and requesting the expert action

$$a^* = \pi_E(s).$$

The expanded dataset therefore contains examples from the learner’s own operating distribution.

At the same time, the temporal representation of the policy can be changed from one-step prediction,

$$p(a_t|o_t),$$

to chunked prediction,

$$p(a_{t:t+H-1}|o_t).$$

This gives the model access to short-horizon trajectory structure and makes it possible to represent multiple coherent future behaviors.

The broader goal of imitation learning can therefore be viewed as making the learner’s rollout distribution resemble the expert’s:

$$p_{\pi_\theta}(\tau) \approx p_{\pi_E}(\tau).$$

Behavior cloning attempts to achieve this indirectly through local supervised imitation.

Dagger improves the match between training and deployment state distributions.

Action chunking improves the representation of temporal action structure.

Distributional action models address the possibility that multiple different future trajectories may all be valid.

5.23 Summary

Behavior cloning formulates imitation learning as supervised learning on expert observation–action pairs. It is simple and effective when training and deployment observations are similar.

The difficulty is that a control policy changes the states it encounters. Small prediction errors can push the system away from the expert state distribution, producing covariate shift and compounding error.

Dagger addresses this problem by collecting states visited by the learner and obtaining expert labels for those states. This teaches the policy how to behave not only under nominal expert conditions but also in states created by its own mistakes.

The trajectory-distribution perspective explains why local imitation error is not sufficient to characterize policy quality. Ultimately, a policy induces a distribution over complete trajectories,

$$p_\pi(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi(a_t|s_t)p(s_{t+1}|s_t, a_t),$$

and the deployment behavior of interest is determined by this rollout distribution.

Robot behavior is also frequently multimodal: several distinct action sequences may successfully solve the same task. Deterministic regression can average these modes and produce an invalid action.

Action chunking addresses temporal representation by predicting

$$A_t = [a_t, \dots, a_{t+H-1}]$$

rather than a single action. This allows the policy to model short-horizon trajectory structure, reduces myopic behavior, and provides a natural representation for generative action models.

Chunked control is commonly combined with receding-horizon execution and temporal ensembling to preserve feedback and improve smoothness.

The concepts therefore address complementary aspects of imitation learning:

behavior cloning $\rightarrow$ learn expert actions,

DAgger $\rightarrow$ reduce learner-expert state-distribution mismatch,

and

action chunking $\rightarrow$ represent coherent short-horizon behavior.

Together, they provide a useful framework for understanding how modern robot policies move from simple supervised imitation toward robust trajectory-level behavior.

6 Reinforcement Learning Fundamentals

Reinforcement learning studies sequential decision-making problems in which an agent interacts with an environment, receives feedback in the form of rewards, and attempts to learn behavior that maximizes long-term performance. Unlike supervised learning, where the desired output is usually provided directly for each input, reinforcement learning must infer which actions are desirable from their consequences over time.

This temporal dependence is the defining difficulty of reinforcement learning. An action may produce little immediate reward but place the system in a state from which large future rewards become possible. Conversely, an action that appears beneficial in the short term may lead to poor outcomes later. Reinforcement learning therefore requires reasoning about both immediate and delayed consequences.

A useful progression is

MDP/POMDP $\rightarrow$ value functions $\rightarrow$ Bellman equations $\rightarrow$ value-based and policy-based methods $\rightarrow$ actor-critic

Algorithms such as Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) can then be understood as particular actor-critic methods with different assumptions about data collection and optimization.

6.1 Markov Decision Processes

The standard mathematical model for reinforcement learning is the *Markov Decision Process* (MDP). An MDP is usually written as

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma),$$

where

- $\mathcal{S}$ is the state space,
- $\mathcal{A}$ is the action space,

- $P(s'|s, a)$ is the transition distribution,
- $R(s, a, s')$ defines the reward,
- $\gamma \in [0, 1)$ is the discount factor.

At time t , the agent observes the current state s_t , chooses an action a_t , receives reward r_t , and transitions to a new state s_{t+1} :

$$s_t \rightarrow a_t \rightarrow (r_t, s_{t+1}).$$

A stochastic policy is a conditional distribution

$$\pi(a|s),$$

and actions are sampled according to

$$a_t \sim \pi(\cdot|s_t).$$

The environment then evolves according to

$$s_{t+1} \sim P(\cdot|s_t, a_t).$$

The policy and transition dynamics together induce a distribution over complete trajectories of states and actions.

6.2 Return and the Reinforcement-Learning Objective

The goal of the agent is not generally to maximize only the immediate reward r_t . Instead, it attempts to maximize the cumulative discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

The discount factor γ controls the relative importance of future rewards. If γ is close to zero, the agent behaves myopically and emphasizes immediate reward. If γ is close to one, future outcomes matter strongly.

The objective of reinforcement learning is therefore commonly written as

$$J(\pi) = \mathbb{E}_{\pi}[G_0].$$

The expectation is necessary because trajectories may be stochastic due to randomness in the policy, the environment, or both.

6.3 The Markov Property

The defining assumption of an MDP is the Markov property:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t).$$

This states that the current state s_t contains all information needed to predict the distribution of the next state once the current action is known.

The Markov property does not mean that history is irrelevant in the physical system. Rather, it means that the state representation already summarizes the relevant history.

For example, consider a moving robot. If its state contains only position,

$$s_t = q_t,$$

then the future may depend on an unobserved velocity, and the representation may not be Markov. A richer state might be

$$s_t = (q_t, \dot{q}_t, \text{object poses, object velocities}).$$

Whether a representation is Markov therefore depends partly on which variables are included in the state.

6.4 Partially Observable Decision Processes

Physical robots rarely observe the full state of the world directly. Cameras may be occluded, object velocities may be unknown, and some relevant quantities may not be sensed at all.

Such problems are naturally modeled as *Partially Observable Markov Decision Processes* (POMDPs).

The underlying environment still has a latent state s_t , but the agent receives an observation o_t generated according to an observation model

$$o_t \sim O(\cdot | s_t).$$

A policy that acts only on the current observation has the form

$$\pi(a_t | o_t).$$

However, a single observation may not contain enough information to make a good decision. A more general policy can condition on observation and action history:

$$\pi(a_t \mid o_{\leq t}, a_{< t}).$$

This is especially important in robotics. A single image may reveal an object’s position but not whether it is moving left or right. Several consecutive frames can provide the missing temporal information.

6.5 Belief States

In a POMDP, an ideal decision-maker maintains a *belief state*,

$$b_t(s) = P(s_t = s \mid o_{\leq t}, a_{< t}).$$

The belief state is a probability distribution over possible hidden states given the available history.

Although explicitly maintaining such a distribution can be difficult in high-dimensional systems, modern neural policies often learn approximate latent state estimators. Recurrent neural networks, Transformers, frame stacks, and latent-state models can all use history to construct an internal representation that serves a similar role.

Thus, in embodied systems, it is often useful to interpret a history encoder as a learned approximation to belief-state estimation.

6.6 State and Action Value Functions

A central idea in reinforcement learning is to assign numerical values to states and actions.

The *state-value function* of policy π is

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid s_t = s].$$

It represents the expected future return when the agent starts in state s and subsequently follows policy π .

The *action-value function* is

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid s_t = s, a_t = a].$$

It represents the expected return after taking action a in state s and then following π .

The difference is therefore

$$V^\pi(s) = \text{value of being in state } s,$$

whereas

$$Q^\pi(s, a) = \text{value of taking action } a \text{ in state } s.$$

These functions convert the long-term sequential objective into quantities that can be estimated from local transitions.

6.7 Bellman Equations

Value functions satisfy recursive relationships known as Bellman equations.

For a fixed policy π ,

$$V^\pi(s) = \mathbb{E}_{a \sim \pi, s' \sim P} [r(s, a, s') + \gamma V^\pi(s')].$$

This equation follows directly from separating the current reward from all future rewards.

The same idea for the action-value function gives

$$Q^\pi(s, a) = \mathbb{E} [r + \gamma \mathbb{E}_{a' \sim \pi} Q^\pi(s', a')].$$

The conceptual structure is

$$\text{current value} = \text{immediate reward} + \text{discounted future value}.$$

Bellman equations are fundamental because they replace a potentially long-horizon return with a one-step recurrence.

6.8 Bellman Optimality

The optimal state- and action-value functions are denoted by

$$V^*(s)$$

and

$$Q^*(s, a).$$

For the optimal action-value function,

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \right].$$

The maximization expresses the assumption that the agent behaves optimally after reaching s' . Once Q^* is known, an optimal deterministic policy can be recovered using

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

This relationship motivates value-based reinforcement-learning methods such as Q-learning.

6.9 Temporal-Difference Learning

Bellman equations also motivate *temporal-difference* (TD) learning.

Suppose the agent observes a transition

$$(s_t, a_t, r_t, s_{t+1}).$$

A one-step Q-learning target is

$$y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a').$$

The temporal-difference error is

$$\delta_t = y_t - Q(s_t, a_t).$$

The current estimate is therefore compared with a bootstrapped estimate containing the immediate reward and the value of the next state.

The term *bootstrapping* refers to using one learned estimate to construct a target for another learned estimate. This makes TD methods efficient, but it can also introduce instability if the estimates are inaccurate.

6.10 Q-Learning

Q-learning attempts to learn the optimal action-value function directly.

In the tabular setting, its update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right],$$

where α is the learning rate.

The term in brackets is exactly the TD error.

Once the Q-function has been learned, the policy is

$$\pi(s) = \arg \max_a Q(s, a).$$

Q-learning is an *off-policy* algorithm. The action actually taken during data collection may come from an exploratory behavior policy, while the learning target uses the greedy action

$$\max_{a'} Q(s', a').$$

The behavior policy and target policy therefore need not be the same.

This makes Q-learning compatible with replay buffers and previously collected experience.

6.11 Deep Q-Networks

When the state space is too large for a table, the Q-function can be approximated by a neural network:

$$Q_\theta(s, a).$$

A Deep Q-Network (DQN) minimizes a Bellman-error objective of the form

$$\mathcal{L}(\theta) = (Q_\theta(s_t, a_t) - y_t)^2,$$

where

$$y_t = r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a').$$

The parameters θ^- belong to a separate *target network* that is updated more slowly than the primary network.

The purpose of the target network is to reduce instability caused by using a rapidly changing function approximator to define its own targets.

6.12 The Difficulty of Continuous Actions

Value-based methods are particularly natural when the action space is small and discrete.

For example,

$$\mathcal{A} = \{\text{left, right, forward, stop}\}$$

can be searched explicitly.

Robotic control often instead requires continuous actions such as

$$a = [\tau_1, \dots, \tau_7] \in \mathbb{R}^7.$$

In this case, evaluating

$$\max_a Q(s, a)$$

may require solving a continuous optimization problem at every decision step.

This difficulty motivates methods that represent the policy directly instead of recovering it through maximization of a value function.

6.13 Policy Gradient Methods

Policy-gradient methods parameterize the policy directly:

$$\pi_\theta(a|s).$$

The objective is

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)].$$

The policy-gradient theorem gives

$$\nabla_\theta J = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t) Q^\pi(s_t, a_t)].$$

The term

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

indicates how the parameters should change to make the sampled action more or less probable. The value term determines whether that probability should increase or decrease.

Actions associated with high return receive positive reinforcement, while actions associated with poor return become less likely.

6.14 REINFORCE

The simplest Monte Carlo policy-gradient algorithm is REINFORCE. It uses the observed return directly:

$$\nabla_{\theta} J \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t.$$

Although conceptually simple, this estimator can have high variance.

For example, a trajectory may receive high total reward even though only a few actions were responsible for the success. Weighting every action by a noisy trajectory return makes learning statistically inefficient.

This motivates the use of baselines and learned value functions.

6.15 The Advantage Function

A common variance-reduction technique is to compare an action with the expected value of the state in which it was selected.

The *advantage function* is

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

It measures whether an action is better or worse than the policy's usual expected outcome from that state.

If

$$A^{\pi}(s, a) > 0,$$

the action performed better than expected and should become more probable.

If

$$A^{\pi}(s, a) < 0,$$

the action performed worse than expected and should become less probable.

The policy gradient can therefore be written as

$$\nabla_{\theta} J = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a | s) A^{\pi}(s, a)].$$

Subtracting a suitable baseline does not change the expected gradient, but can substantially reduce its variance.

6.16 Actor–Critic Methods

Actor–critic methods combine explicit policy learning with value-function estimation.

The *actor*

$$\pi_{\theta}(a|s)$$

represents the policy and determines which action to take.

The *critic*

$$V_{\phi}(s)$$

or

$$Q_{\phi}(s, a)$$

estimates how desirable the actor’s states or actions are.

The actor can therefore be viewed as the component that makes decisions, while the critic provides a learned evaluation signal.

A typical actor update contains a term such as

$$\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \hat{A}_t,$$

where $\hat{A}_t$ is an estimate of the advantage.

The critic is trained by minimizing an error such as

$$(V_{\phi}(s_t) - y_t)^2.$$

Actor–critic methods are especially important for continuous-control problems because they avoid explicitly maximizing $Q(s, a)$ over all possible actions at every timestep.

6.17 Advantage Estimation

One simple estimate of the advantage is the one-step TD error:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t).$$

This estimator has relatively low variance but may be biased when the learned value function is inaccurate.

A widely used alternative is *Generalized Advantage Estimation* (GAE):

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots.$$

Equivalently,

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.$$

The parameter λ controls a bias–variance tradeoff.

Smaller values place more emphasis on short-horizon bootstrapped estimates, whereas values closer to one incorporate longer-horizon return

7 Data Sources and Dataset Design for Robot Learning

The performance of a learned robotic system depends not only on the model architecture and optimization procedure, but also on the data from which the model is trained. In robot learning, this dependence is particularly strong because the data distribution determines which states, actions, environments, and failure modes the policy has had an opportunity to observe. A highly expressive policy cannot reliably infer behavior in regions of the state space that are poorly represented in its training set.

Robot-learning datasets may be collected from several sources, including human teleoperation, autonomous robot interaction, human video, simulation, and other synthetic processes. These sources differ in what kinds of supervision they provide. Teleoperation gives direct correspondence between observations and robot actions. Human video offers large-scale information about task semantics, but usually lacks robot-specific control labels. Simulation provides inexpensive and precisely labeled experience, although the simulated distribution may differ from the physical world. Replay buffers and dataset sampling strategies further determine which of these experiences are emphasized during optimization.

Consequently, dataset design is not simply a matter of collecting as many examples as possible. It involves deciding which experiences should be collected, how they should be represented, and how frequently they should appear during training.

7.1 Demonstrations and Trajectories

A common unit of data in robot learning is the *trajectory*. A trajectory records the evolution of observations and actions over time and may be written as

$$\tau = (o_0, a_0, o_1, a_1, \dots, o_T, a_T),$$

where o_t denotes the observation available to the agent at time t , and a_t denotes the corresponding action. If the complete physical state of the system is available, the trajectory may instead be written using s_t :

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T).$$

Depending on the learning problem, additional quantities may also be associated with the trajectory. These can include rewards r_t , task instructions ℓ , success indicators, contact labels, or other auxiliary signals.

A demonstration dataset containing N trajectories can therefore be written as

$$\mathcal{D} = \{\tau_1, \tau_2, \dots, \tau_N\}.$$

The term *demonstration* is often used when a trajectory is intended to illustrate useful or task-relevant behavior. Importantly, however, a demonstration is not necessarily optimal. Human demonstrations may contain hesitation, unnecessary motions, imperfect corrections, or complete failures. A learned policy therefore tends to reproduce the statistical structure of the demonstrated behavior rather than some abstract notion of the best possible behavior.

This distinction is important in imitation learning. If undesirable behaviors appear frequently in the dataset, a sufficiently powerful imitation model may learn them along with the desirable ones.

7.2 Behavior Cloning from Demonstrations

The simplest use of demonstrations is behavior cloning. In behavior cloning, a policy is trained as a supervised learning model that predicts the demonstrated action from the current observation.

Let

$$\pi_{\theta}(a \mid o)$$

denote a policy parameterized by θ . For deterministic continuous actions, a common training objective is

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(o,a) \sim \mathcal{D}} \left[\|\pi_{\theta}(o) - a\|_2^2 \right].$$

The policy is therefore encouraged to produce an action close to the demonstrated action.

For stochastic or discrete-action policies, behavior cloning is commonly formulated as maximum likelihood:

$$\mathcal{L}_{\text{BC}}(\theta) = -\mathbb{E}_{(o,a) \sim \mathcal{D}} [\log \pi_{\theta}(a \mid o)].$$

In both cases, the training signal comes directly from the demonstration dataset. This makes behavior cloning simple and stable, but it also means that the resulting policy is strongly shaped by the quality and coverage of that dataset.

7.3 Dataset Quality, Diversity, and Coverage

Three closely related properties are useful when reasoning about robot-learning datasets: *quality*, *diversity*, and *coverage*.

Dataset quality describes how desirable or correct the recorded actions are. A dataset containing smooth, successful, and deliberate demonstrations usually provides a cleaner supervision signal than one dominated by operator errors or failed trials.

Diversity describes variation across the dataset. This may include variation in objects, lighting, camera viewpoints, task instructions, initial conditions, robot poses, environments, and strategies for solving a task. Diversity is particularly important when the learned policy will be expected to generalize beyond a narrow training setup.

Coverage refers to the extent to which the relevant state and action spaces are represented. A dataset may contain a large number of examples while still having poor coverage. For example, a manipulation dataset may contain millions of observations of successful grasps but almost no examples of recovery after a missed grasp.

The three properties are related but not interchangeable. A small dataset can have high quality but insufficient coverage. Conversely, a very large dataset can have broad coverage but contain low-quality demonstrations. Generalization depends on the combination of these properties rather than on dataset size alone.

7.4 Teleoperation as a Source of Robot Data

Teleoperation is one of the most direct methods for collecting robot demonstrations. A human operator controls the physical or simulated robot using an interface such as a joystick, spacemouse, VR controller, motion-capture system, or leader robot. While the operator performs the task, the system records the robot's observations and actions.

For a manipulation system, the observation might include an RGB image I_t , robot joint configuration q_t , and gripper state g_t :

$$o_t = (I_t, q_t, g_t).$$

The corresponding action might be expressed as a relative end-effector command,

$$a_t = (\Delta x_t, \Delta y_t, \Delta z_t, \Delta r_t, g_{t+1}),$$

where the translational and rotational increments define the desired motion and g_{t+1} controls the gripper.

The principal advantage of teleoperation is that the robot observation and the executed action are naturally aligned in time. The resulting pairs

$$(o_t, a_t)$$

can therefore be used directly for behavior cloning.

This property distinguishes teleoperation from ordinary human video. A video of a person performing a task may reveal what happened, but it does not directly specify which actuator commands a robot should produce.

7.5 Noise and Bias in Teleoperation

Teleoperation should not be treated as a perfect source of expert behavior. Human operators often overshoot targets, pause unnecessarily, make corrective motions, or compensate for delays in the control interface. Some behaviors may reflect the peculiarities of the teleoperation device rather than the strategy that an autonomous robot should ideally execute.

As a result, the teleoperation action distribution

$$p_{\text{teleop}}(a \mid o)$$

may differ from the action distribution desired at deployment.

Practical datasets are therefore often processed before training. Trajectories may be segmented into individual tasks, synchronized across sensors, filtered for quality, labeled as successful or unsuccessful, or smoothed to reduce control noise. The preprocessing stage can substantially affect the final policy because it determines which parts of the recorded experience remain in the training distribution.

7.6 Learning from Human Video

Human video provides a very different type of supervision. Large-scale video collections may contain people interacting with objects, manipulating tools, opening doors, preparing food, and performing many other tasks relevant to robotics.

Such data contain rich semantic information. From a video of someone opening a drawer, a model may learn that the handle is the relevant interaction point, that grasping precedes pulling, and that the desired final state is an open drawer.

However, human video does not normally provide robot control labels. It does not specify the robot’s joint configuration,

$$[q_1, \dots, q_n],$$

its actuator torques,

$$[\tau_1, \dots, \tau_n],$$

or the precise end-effector commands required for execution.

Human video is therefore more naturally interpreted as supervision about the structure and semantics of a task than as direct supervision of robot motor commands.

7.7 The Embodiment Gap

The difficulty of transferring behavior from humans to robots is often described as the *embodiment gap*. Humans and robots differ in morphology, kinematics, sensing, control frequency, force capability, reachability, and compliance. Even when a human and robot accomplish the same semantic task, the physical trajectories used to do so may be very different.

For example, human video may reveal that opening a drawer involves the sequence

locate handle $\rightarrow$ grasp handle $\rightarrow$ pull outward.

This sequence captures the task structure. It does not determine the robot-specific grasp pose, joint configuration, force profile, or motion trajectory.

It is therefore useful to distinguish two forms of knowledge:

human data $\rightarrow$ what should happen,

and

robot data $\rightarrow$ how this robot can make it happen.

This distinction explains why large-scale human data and comparatively small robot datasets can be complementary rather than redundant. Human data can supply broad semantic priors, while robot data ground those priors in the robot’s own action space.

7.8 Synthetic Data and Simulation

Synthetic data are generated by an artificial process rather than collected directly from real-world human or robot interaction. In robotics, physics simulation is one of the most important synthetic data sources.

A simulator can produce complete trajectories together with quantities that may be difficult or expensive to measure in the physical world. These can include RGB images, depth maps, segmentation masks, contact states, forces, rewards, and exact object poses.

For example, the simulator may directly provide the six-degree-of-freedom pose

$$T_i \in SE(3)$$

of every object in the environment. In the real world, obtaining the same quantity would require perception, motion capture, or another estimation system.

Simulation is therefore especially valuable when precise labels are required at large scale.

Another advantage is that simulated interaction can be parallelized. Thousands of simulated environments may generate experience simultaneously, allowing data collection at rates that would be impractical with physical robots. Simulation also makes it possible to create rare or dangerous scenarios without risking hardware.

7.9 The Sim-to-Real Gap

The usefulness of simulation is limited by the fact that simulated and physical environments are not identical.

Let

$$p_{\text{sim}}(x)$$

denote the distribution of observations and experiences produced in simulation, and

$$p_{\text{real}}(x)$$

the corresponding real-world distribution. In general,

$$p_{\text{sim}}(x) \neq p_{\text{real}}(x).$$

This discrepancy is known as the *sim-to-real gap*.

The gap can arise from inaccuracies in rendering, contact mechanics, actuator dynamics, friction, compliance, sensor noise, latency, and many other factors. Even small modeling errors can become important if a learned policy discovers behavior that exploits simulator-specific properties.

For this reason, simulation is better understood as a source of inexpensive and controllable experience than as a perfect replacement for real-world data.

7.10 Domain Randomization

One common method for reducing sensitivity to simulation inaccuracies is *domain randomization*. Instead of training in a single fixed simulated environment, properties of the simulation are varied across episodes.

Randomized parameters may include lighting, texture, object geometry, camera position, friction, mass, actuator parameters, sensor noise, and initial conditions.

The objective is not necessarily to make any one simulation perfectly realistic. Instead, the goal is to make

$$p_{\text{sim}}(x)$$

broad enough that the physical deployment distribution lies within, or close to, the range of variations encountered during training.

The intuition is that a policy which succeeds under many simulated variations is less likely to depend on a simulator-specific feature that disappears in the real world.

7.11 Other Forms of Synthetic Data

Synthetic data are not limited to physics simulation. Procedural generators can create object configurations, environments, and task variants. Generative models can produce images, language instructions, or candidate trajectories. Language data can also be augmented through paraphrasing.

For instance, an instruction such as

Pick up the red cup.

might be augmented with semantically equivalent instructions such as

Grab the red mug.

or

Lift the red cup from the table.

Such augmentation changes the linguistic distribution seen during training and may improve robustness to phrasing at deployment.

7.12 Replay Buffers

In reinforcement learning, interaction data are often stored in a *replay buffer*. A basic replay buffer contains transitions of the form

$$(s_t, a_t, r_t, s_{t+1}),$$

possibly together with termination indicators, task identities, success labels, or auxiliary sensor information.

Let the replay buffer be denoted by

$$\mathcal{B}.$$

Training samples can then be drawn according to

$$(s, a, r, s') \sim \mathcal{B}.$$

The replay buffer separates data collection from parameter updates. Rather than learning only from the newest transition, the algorithm can repeatedly reuse earlier experience.

7.13 Why Experience Replay Is Useful

Experience replay has two important effects.

First, it improves sample efficiency. A transition obtained from an expensive physical interaction can contribute to multiple gradient updates rather than being discarded after one use.

Second, it reduces temporal correlation between training examples. Consecutive states in a trajectory are usually highly correlated:

$$s_t \approx s_{t+1}.$$

If optimization repeatedly uses consecutive transitions, successive minibatches may contain very similar information. Random sampling from a replay buffer mixes transitions collected at different times and in different episodes, producing a more diverse stochastic gradient estimate.

Replay buffers are especially natural in off-policy algorithms such as Q-learning and Soft Actor-Critic because these methods can learn from experience generated by policies other than the current policy.

7.14 Trajectory Replay

For sequence models, a single transition may not contain enough context. Recurrent policies, Transformers, and action-chunking policies often require a temporal window.

The replay unit may therefore be a subsequence such as

$$\tau_{t:t+H} = (o_t, a_t, \dots, o_{t+H}, a_{t+H}).$$

Sampling entire windows preserves the temporal dependencies needed for learning motion, velocity, multi-step behaviors, and other sequential structure.

7.15 Replay Buffers and Offline Datasets

A replay buffer should be distinguished from an offline dataset.

A replay buffer normally changes while learning proceeds. If $\mathcal{B}_t$ denotes its contents at training time t , then

$$\mathcal{B}_t \rightarrow \mathcal{B}_{t+1}$$

as new experience is added and possibly old experience is removed.

An offline dataset, by contrast, is fixed before training begins:

$$\mathcal{D}_{\text{offline}} = \text{constant}.$$

The policy cannot collect new environment interactions to correct weaknesses in the dataset. Consequently, the quality and coverage of an offline dataset are particularly important.

7.16 Dataset Imbalance

Large robot datasets are rarely balanced. Some tasks may appear millions of times, whereas others may have only a few thousand examples. Similarly, long stretches of ordinary free-space motion may dominate rare events such as grasp transitions, collisions, corrective maneuvers, or recovery behavior.

Suppose task k contributes n_k examples. If individual examples are sampled uniformly from the raw dataset, then the probability that a training example belongs to task k is

$$P(k) = \frac{n_k}{\sum_j n_j}.$$

Consequently, tasks with more recorded examples contribute proportionally more gradient updates.

This is not necessarily undesirable. If the raw dataset accurately reflects the distribution encountered at deployment, such sampling may be appropriate. However, it can cause rare tasks or rare but safety-critical events to contribute very little to the training objective.

7.17 Task Balancing

One alternative is to sample tasks uniformly:

$$P(k) = \frac{1}{K},$$

where K is the number of tasks.

This removes the influence of dataset size, but may give excessive weight to tasks represented by very small datasets.

A useful interpolation between raw-frequency sampling and uniform task sampling is

$$P(k) \propto n_k^\alpha, \quad 0 < \alpha < 1.$$

When

$$\alpha = 1,$$

sampling follows the raw dataset frequency. As

$$\alpha \rightarrow 0,$$

the sampling distribution approaches uniform weighting across tasks.

The parameter α therefore controls how aggressively large tasks are downweighted relative to small tasks.

7.18 Balancing Rare Events

Balancing may also be performed according to event type rather than task identity.

Consider a dataset in which 95% of episodes are successful and only 5% contain failures. If samples are drawn uniformly, the optimizer sees relatively few examples of how the robot behaves near failure states.

However, these states may be precisely the ones in which robust recovery behavior is most important.

One may therefore oversample examples containing

- failures,
- collisions,
- grasp transitions,
- contact events,
- corrective actions,
- recovery behavior.

The appropriate sampling distribution depends on the objective. Rare events need not be sampled according to their natural frequency if their consequences at deployment are disproportionately important.

7.19 The Effective Training Distribution

Sampling and weighting modify the distribution actually seen by the optimizer.

Suppose the collected dataset follows

$$p_{\text{data}}(x).$$

If an example x is assigned sampling weight $w(x)$, then the effective training distribution is proportional to

$$p_{\text{train}}(x) \propto w(x)p_{\text{data}}(x).$$

This equation is important because it shows that dataset balancing is not merely an implementation detail. It changes the statistical objective being optimized.

The model is not trained on the raw dataset distribution unless

$$w(x) = \text{constant}.$$

Every choice of oversampling, undersampling, prioritization, or task reweighting changes which regions of the experience space contribute most strongly to learning.

7.20 Training Distribution and Deployment Distribution

Ideally, the training data should provide sufficient support for the conditions encountered during deployment.

Let

$$x \sim p_{\text{train}}(x)$$

denote training experiences and

$$x \sim p_{\text{deploy}}(x)$$

denote deployment experiences.

If the deployment distribution contains states that are rarely or never represented under p_{train} , the policy must operate out of distribution.

This observation suggests an important principle of robot dataset design: coverage should include not only nominal task execution, but also states created by disturbances, mistakes, environmental variation, and recovery.

The desired training distribution is therefore not always identical to the naturally collected distribution. Dataset design often requires deliberately increasing representation of states that are rare in collected data but important for robust deployment.

7.21 Combining Data Sources

Modern robot-learning systems often combine multiple data sources because no single source provides all of the required information.

A useful conceptual decomposition is

Human video $\rightarrow$ semantic and task knowledge,
teleoperation $\rightarrow$ robot-specific action supervision,
simulation $\rightarrow$ large-scale controlled variation,
autonomous interaction $\rightarrow$ states induced by deployed policies.

These sources may then be filtered, segmented, labeled, and sampled to construct the final training distribution.

The resulting pipeline can be written abstractly as

$$\text{Raw Experience} \rightarrow \text{Curation} \rightarrow \text{Sampling} \rightarrow p_{\text{train}} \rightarrow \text{Optimization} \rightarrow \pi_{\theta}.$$

This perspective highlights an important fact: the learner never sees “the dataset” in an abstract sense. It sees a sequence of minibatches generated by the collection and sampling procedure.

7.22 Dataset Design as Part of the Learning Algorithm

Dataset design is sometimes treated as separate from model design, but in large-scale robot learning the two are closely coupled.

Changing the model architecture changes which forms of data can be exploited. For example, temporal models can make use of trajectory windows that would be invisible to a single-frame policy. Conversely, changing the dataset distribution changes which behaviors the model is encouraged to represent.

A policy trained mainly on successful nominal behavior may become highly accurate in nominal states but fragile under disturbance. A policy trained on a more diverse collection containing corrective actions and failure recovery may have slightly more difficult optimization but greater deployment robustness.

Thus, the final learned behavior depends jointly on

model capacity, training objective, data distribution.

In practice, these should be designed together.

7.23 Summary

Robot-learning datasets are composed of trajectories containing observations, actions, and potentially rewards, task descriptions, and auxiliary labels. Demonstrations provide direct supervision for imitation learning, but their usefulness depends on their quality and coverage.

Teleoperation is particularly valuable because it yields robot-specific observation-action pairs, although operator noise and interface bias can enter the data. Human video provides vastly larger-scale semantic information but suffers from an embodiment gap because human actions do not directly map to robot control commands.

Simulation and other synthetic sources provide inexpensive and precisely labeled data, but introduce a sim-to-real distribution gap. Domain randomization attempts to improve transfer by deliberately broadening the simulation distribution.

Replay buffers improve sample efficiency and reduce temporal correlation by reusing previously collected interaction data. Finally, dataset balancing determines the effective distribution presented to the optimizer. If raw data follow $p_{\text{data}}(x)$ and examples are sampled with weight $w(x)$, then the learner effectively trains under

$$p_{\text{train}}(x) \propto w(x)p_{\text{data}}(x).$$

The central lesson is that the behavior of a learned robot is shaped not simply by the number of examples available, but by the distribution of experience that reaches the optimizer. Dataset quality, diversity, coverage, and sampling strategy are therefore fundamental components of robot-policy design.

8 Evaluation Under Distribution Shift

A learned robotic policy is useful only to the extent that it performs reliably on the distribution of situations encountered at deployment. Evaluating such a policy therefore requires more than measuring average performance on a held-out subset of the same dataset used for training. In embodied systems, changes in environment, object appearance, robot state, camera viewpoint, task specification, or dynamics can move the system away from the training distribution. A useful evaluation methodology must explicitly characterize this distribution shift and measure both nominal task performance and robustness to unfamiliar conditions.

8.1 Training and Test Distributions

Let the training data be drawn from a distribution

$$(x, y) \sim p_{\text{train}}(x, y),$$

where x denotes the model input and y denotes the target output. In robot learning, x may contain observations, language instructions, proprioceptive state, or complete trajectory histories, while y may represent actions, action chunks, rewards, or task outcomes.

Evaluation data are drawn from

$$(x, y) \sim p_{\text{test}}(x, y).$$

The simplest evaluation setting assumes

$$p_{\text{test}}(x, y) \approx p_{\text{train}}(x, y).$$

This is commonly referred to as an *independent and identically distributed*, or IID, evaluation setting. The test examples are different from the training examples, but they are assumed to originate from approximately the same underlying distribution.

For example, suppose a manipulation dataset contains mugs placed randomly on a table, with object position sampled according to

$$(x_{\text{mug}}, y_{\text{mug}}) \sim p_{\text{train}}.$$

A conventional test set may use previously unseen mug placements sampled from the same distribution. Such a test measures generalization to unseen examples but does not necessarily measure generalization to qualitatively new conditions.

In deployment, it is common for

$$p_{\text{deployment}} \neq p_{\text{train}}.$$

This discrepancy is called *distribution shift*. Distribution shift may occur in the visual inputs, robot dynamics, task distribution, action consequences, or combinations of these factors.

A useful distinction is between changes in the input distribution and changes in the underlying relationship between inputs and desired outputs. In classical covariate shift,

$$p_{\text{train}}(x) \neq p_{\text{test}}(x),$$

while approximately

$$p_{\text{train}}(y|x) = p_{\text{test}}(y|x).$$

For example, a robot may encounter a mug of an unfamiliar color, even though the correct grasping behavior remains essentially unchanged.

A more difficult form of distribution shift occurs when

$$p_{\text{train}}(y|x) \neq p_{\text{test}}(y|x).$$

This can occur if system dynamics or task semantics change. For instance, the same commanded gripper motion may have different consequences if the robot carries a heavier payload or if actuator dynamics differ from those present during training.

8.2 In-Distribution and Out-of-Distribution Evaluation

An input is generally described as *in-distribution* if it is reasonably consistent with the support of the training distribution. An *out-of-distribution* input lies outside, or in a sparsely represented region of, that distribution.

If

$$x \sim p_{\text{train}}(x),$$

then x is nominally in-distribution. If instead

$$x \sim p_{\text{OOD}}(x),$$

where p_{OOD} differs significantly from the training distribution, the example is considered out-of-distribution.

Out-of-distribution evaluation should not be interpreted as a single test. There are many possible forms of OOD shift, and a system may generalize well to some while failing catastrophically on others.

For a robotic manipulation system, possible OOD dimensions include

- novel object instances,
- novel object categories,
- unfamiliar textures or colors,
- different illumination,
- changes in camera viewpoint,
- novel backgrounds or clutter,
- unseen object poses,
- unfamiliar language instructions,
- changes in payload or friction,
- sensor corruption,
- actuator delay,
- changes in robot embodiment,
- combinations of several simultaneous shifts.

It is therefore useful to define explicit evaluation distributions. For example,

$$p_{\text{ID}}$$

may contain familiar object categories and environments, while

$$p_{\text{OOD,obj}}$$

contains unseen object instances,

$p_{\text{OOD,view}}$

contains novel camera viewpoints, and

$p_{\text{OOD,dyn}}$

contains altered system dynamics.

This decomposition provides considerably more information than reporting a single ‘OOD accuracy’ or ‘OOD success rate.’

8.3 Generalization Across Increasing Distribution Shift

One useful evaluation methodology is to construct several levels of distribution shift. For example,

D_0 = nominal test distribution,

D_1 = moderate visual variation,

D_2 = unseen object configurations,

D_3 = combined visual and dynamics shift.

The policy can then be evaluated using

$$S(D_i),$$

where S denotes task success under distribution D_i .

The degradation

$$\Delta_i = S(D_0) - S(D_i)$$

measures sensitivity to a particular degree of distribution shift.

A model with slightly lower nominal performance but smaller degradation under shift may be preferable to a model that achieves near-perfect IID performance but fails sharply outside the training distribution.

8.4 Task Success Metrics

For many robotic tasks, the most interpretable metric is the *task success rate*. Given N evaluation episodes, define

$$s_i = \begin{cases} 1, & \text{if episode } i \text{ succeeds,} \\ 0, & \text{otherwise.} \end{cases}$$

The empirical success rate is

$$\hat{S} = \frac{1}{N} \sum_{i=1}^N s_i.$$

If 83 out of 100 manipulation attempts succeed, then

$$\hat{S} = 0.83.$$

Success criteria should be defined operationally before evaluation. For example, “pick up the object” may require that the object remain above the table for at least a specified duration rather than merely making temporary contact with the gripper.

Binary task success is often insufficient by itself. Additional metrics can reveal how and why the system succeeds or fails. Depending on the task, useful quantities include

- completion time,
- number of control steps,
- path length,
- final pose error,
- grasp stability,
- collision count,
- peak contact force,
- energy consumption,
- number of replanning events,
- number of human interventions.

For a target pose T^* and achieved pose T , translational error may be measured as

$$e_p = \|p - p^*\|_2.$$

Rotational error may be measured using the relative rotation

$$R_{\text{err}} = R^{*T} R$$

and the corresponding angle

$$e_R = \cos^{-1} \left(\frac{\text{tr}(R_{\text{err}}) - 1}{2} \right).$$

These continuous metrics provide useful information even when the final task result is represented by a binary success variable.

8.5 Conditional Success Rates

An overall success rate can hide substantial differences across task categories. Suppose a policy is evaluated on several task groups c . The conditional success rate is

$$S_c = P(\text{success} \mid c).$$

For example,

$$S_{\text{pick}}, \quad S_{\text{place}}, \quad S_{\text{drawer}}, \quad S_{\text{pour}}$$

may differ considerably.

The same decomposition can be performed over object categories, environments, robot configurations, or OOD conditions. A model whose average success is high may still have severe blind spots if one subgroup has significantly lower performance.

A useful aggregate metric is the worst-group success rate,

$$S_{\text{worst}} = \min_c S_c.$$

This quantity provides a simple measure of whether performance is concentrated in easy or overrepresented subsets of the evaluation distribution.

8.6 Robustness

Robustness refers to the degree to which performance is preserved under perturbations, uncertainty, noise, or distribution shift.

Let

$$f_{\theta}(x)$$

denote the learned policy and let

$$x' = x + \delta$$

represent a perturbed input. A robust policy should ideally produce behavior that remains appropriate for perturbations δ within a relevant set of disturbances.

In robotics, robustness should be evaluated at the system level rather than only through small perturbations of neural network inputs. Relevant perturbations include sensor noise, object displacement, latency, external forces, inaccurate calibration, and imperfect actuation.

For a perturbation magnitude ϵ , define an evaluation distribution

$$p_{\epsilon}(x)$$

containing environments with perturbations of approximately that magnitude. The resulting robustness curve can be expressed as

$$S(\epsilon) = P(\text{success} \mid \epsilon).$$

A robust system exhibits graceful degradation:

$$S(\epsilon)$$

decreases gradually as perturbations become more severe, rather than collapsing immediately after departing from nominal conditions.

A useful summary metric is the relative robustness

$$R(\epsilon) = \frac{S(\epsilon)}{S(0)}.$$

A value

$$R(\epsilon) \approx 1$$

indicates little performance degradation under the specified perturbation.

8.7 Perturbation and Recovery Tests

Closed-loop robotic systems should also be evaluated on their ability to recover from unexpected disturbances.

Consider a manipulation trajectory

$$\tau = (o_0, a_0, \dots, o_T, a_T).$$

At some time t_p , an external perturbation may be introduced, such as moving the target object, perturbing the robot arm, or temporarily occluding the camera.

The evaluation then measures whether the policy can return to a successful trajectory without external assistance.

Possible recovery metrics include

$$P(\text{recovery} \mid \text{perturbation}),$$

time to recovery,

$$T_{\text{recover}},$$

and additional trajectory cost caused by the disturbance,

$$\Delta C = C_{\text{perturbed}} - C_{\text{nominal}}.$$

Recovery tests are especially valuable because a policy may perform well when initialized near expert trajectories but fail to return to the task manifold after small deviations.

8.8 Intervention Rate

For systems deployed with human supervision, task success alone may substantially overestimate autonomy. A robot may complete nearly every episode but require frequent human correction.

Let I_i denote the number of human interventions during episode i . A simple intervention rate per episode is

$$R_{\text{int}} = \frac{1}{N} \sum_{i=1}^N I_i.$$

Alternatively, if total operating time is T ,

$$R_{\text{int,time}} = \frac{N_{\text{interventions}}}{T}.$$

For continuously deployed systems, one may report interventions per hour,

$$R_{\text{int,hour}} = \frac{N_{\text{interventions}}}{T_{\text{hours}}}.$$

Another useful quantity is the fraction of episodes requiring at least one intervention:

$$P_{\text{int}} = \frac{\sum_i \mathbb{I}[I_i > 0]}{N}.$$

Intervention rate is particularly important when comparing partially autonomous systems. Two policies may both achieve

95%

task completion, but one may require a human correction every few minutes while the other runs autonomously for several hours.

8.9 Intervention Severity

Not all interventions are equivalent. A useful evaluation scheme may distinguish between

- minor corrections,
- trajectory resets,
- task restarts,
- safety-critical takeovers.

A weighted intervention cost can be defined as

$$C_{\text{int}} = \sum_{j=1}^M w_j I_j,$$

where I_j denotes the number of interventions of type j and w_j represents their severity.

For example, a brief waypoint correction may have a small cost, while an emergency stop may have a much larger cost.

This provides a better measure of operational autonomy than treating every intervention identically.

8.10 Success Without Intervention

For real autonomous systems, an especially informative metric is the probability of completing a task without human assistance:

$$S_{\text{autonomous}} = P(\text{success} \cap \text{no intervention}).$$

This differs from ordinary task success,

$$S = P(\text{success}),$$

because an episode recovered by human takeover may count as successful operationally but should not be counted as autonomous success.

The distinction becomes increasingly important for long-horizon tasks, where human assistance can conceal substantial weaknesses in the learned policy.

8.11 Long-Horizon Reliability

Per-action or per-subtask success rates can be misleading when tasks contain many sequential stages.

Suppose a system must successfully complete K independent stages, each with success probability

$$p.$$

The complete task success probability is

$$P_{\text{task}} = p^K.$$

For example, if

$$p = 0.98$$

and

$$K = 50,$$

then

$$P_{\text{task}} = 0.98^{50} \approx 0.364.$$

Thus, seemingly strong local reliability may still produce poor long-horizon autonomy.

For this reason, embodied systems should be evaluated not only on individual skills but also on complete multi-stage tasks and extended deployment periods.

A related operational metric is mean time between interventions,

$$\text{MTBI} = \frac{T_{\text{operation}}}{N_{\text{interventions}}}.$$

Larger values indicate longer periods of sustained autonomous operation.

8.12 Robust Evaluation Protocols

A strong evaluation protocol should separate several distinct questions.

First, *in-distribution generalization* asks whether the model performs well on unseen examples sampled from conditions similar to training.

Second, *systematic OOD evaluation* asks how performance changes when one or more factors differ from the training distribution.

Third, *robustness evaluation* introduces controlled disturbances or noise and measures graceful degradation and recovery.

Fourth, *autonomy evaluation* measures how frequently external intervention is required during extended operation.

An evaluation suite might therefore report

$$S_{\text{ID}},$$

$$S_{\text{OOD,obj}},$$

$$S_{\text{OOD,env}},$$

$$S_{\text{OOD,dyn}},$$

$$S_{\text{perturbed}},$$

along with

$$R_{\text{int}}, \quad S_{\text{autonomous}}, \quad \text{MTBI}.$$

Together, these quantities provide a substantially more complete characterization of system behavior than a single average success rate.

8.13 Statistical Uncertainty

Reported evaluation metrics should also include uncertainty. If N independent episodes are evaluated and $\hat{S}$ is the observed success rate, an approximate standard error is

$$\text{SE}(\hat{S}) = \sqrt{\frac{\hat{S}(1 - \hat{S})}{N}}.$$

A model evaluated on only a small number of episodes may therefore have a highly uncertain success estimate.

For example, observing nine successes out of ten trials,

$$\hat{S} = 0.9,$$

does not provide the same statistical evidence as observing 900 successes out of 1000 trials, even though the reported success rate is identical.

This consideration is particularly important in robotics, where physical experiments are expensive and evaluation sets are often small.

8.14 Evaluation as Distribution Design

The central principle is that evaluation is fundamentally a problem of specifying distributions.

Training occurs under some distribution

$$p_{\text{train}},$$

while the system is ultimately required to perform under a deployment distribution

$$p_{\text{deploy}}.$$

An evaluation suite defines one or more intermediate distributions

$$p_{\text{eval}}^{(1)}, p_{\text{eval}}^{(2)}, \dots, p_{\text{eval}}^{(K)}$$

designed to probe relevant aspects of this gap.

A useful evaluation should therefore answer not only

“What is the average success rate?”

but also

“Under which distributions does the policy succeed?”

“How rapidly does performance degrade as conditions change?”

and

“How often must an external agent intervene when the policy encounters unfamiliar states?”

For embodied systems, these questions are often more informative than nominal test-set performance because deployment necessarily exposes the policy to conditions that cannot be represented exhaustively in the training data.

9 Robot Kinematics, Dynamics, and Control

Robot motion is commonly described at two complementary levels. *Kinematics* describes the geometric relationship between joint variables and the position and orientation of the robot, without considering forces or masses. *Dynamics* describes how forces and torques produce motion. Control methods use these models to determine the commands required to achieve desired trajectories, forces, or interactions with the environment.

9.1 Forward and Inverse Kinematics

Let the robot configuration be described by a vector of generalized coordinates

$$q = [q_1, \dots, q_n]^T.$$

For a serial manipulator, these coordinates typically correspond to joint angles or prismatic joint displacements.

Forward kinematics computes the pose of the end effector from the joint configuration:

$$T_{WE} = f(q),$$

where T_{WE} is the rigid-body transformation from the end-effector frame E to the world frame W .

Forward kinematics therefore solves the mapping

$$q \longrightarrow \text{end-effector pose.}$$

Inverse kinematics solves the opposite problem. Given a desired end-effector pose

$$T_{WE}^*,$$

the goal is to determine a configuration q^* satisfying

$$f(q^*) = T_{WE}^*.$$

Inverse kinematics may have zero, one, or several solutions. A desired pose may be unreachable, multiple configurations may produce the same end-effector pose, and redundant robots may possess infinitely many configurations that realize the same task.

For simple manipulators, inverse kinematics can sometimes be solved analytically. For more general robots, numerical optimization is commonly used:

$$q_{k+1} = q_k + \Delta q_k.$$

The update Δq_k can be obtained using the manipulator Jacobian.

9.2 Rigid-Body Transformations and $SE(3)$

The pose of a rigid body consists of a translation and a rotation. It is commonly represented by a homogeneous transformation matrix

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix},$$

where

$$R \in SO(3)$$

is a rotation matrix and

$$p \in \mathbb{R}^3$$

is a translation vector.

The group

$$SO(3) = \{R \in \mathbb{R}^{3 \times 3} \mid R^T R = I, \det(R) = 1\}$$

contains all valid three-dimensional rotations.

Rigid-body poses belong to the special Euclidean group

$$SE(3) = \left\{ \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} : R \in SO(3), p \in \mathbb{R}^3 \right\}.$$

Homogeneous transformations are particularly useful because rotations and translations can be composed using ordinary matrix multiplication. If

$$T_{AB}$$

describes frame B relative to frame A , and

$$T_{BC}$$

describes frame C relative to frame B , then

$$T_{AC} = T_{AB} T_{BC}.$$

The inverse transformation is

$$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}.$$

These identities are fundamental when transforming camera measurements, end-effector poses, object poses, and sensor data between coordinate frames.

9.3 Rotation Representations and Quaternions

A three-dimensional rotation may be represented using a rotation matrix, Euler angles, axis-angle coordinates, or a quaternion.

A unit quaternion has the form

$$q = [w \ x \ y \ z]^T,$$

with

$$w^2 + x^2 + y^2 + z^2 = 1.$$

Equivalently,

$$q = \begin{bmatrix} \cos(\theta/2) \\ u \sin(\theta/2) \end{bmatrix},$$

where u is the unit rotation axis and θ is the rotation angle.

Quaternions are widely used because they avoid the singularities associated with Euler angles and require fewer parameters than a full rotation matrix. However, they provide a double representation:

$$q \quad \text{and} \quad -q$$

describe the same physical rotation.

9.4 The Manipulator Jacobian

The Jacobian relates joint velocity to end-effector velocity.

Let the end-effector spatial velocity, or twist, be

$$\xi = \begin{bmatrix} v \\ \omega \end{bmatrix},$$

where v is translational velocity and ω is angular velocity.

The manipulator Jacobian satisfies

$$\boxed{\xi = J(q)\dot{q}.$$

Thus,

$$J(q) \in \mathbb{R}^{6 \times n}$$

maps joint velocities to Cartesian velocity.

For a revolute joint with axis z_i , joint origin p_i , and end-effector position p_E , the Jacobian column is

$$J_i = \begin{bmatrix} z_i \times (p_E - p_i) \\ z_i \end{bmatrix}.$$

For a prismatic joint,

$$J_i = \begin{bmatrix} z_i \\ 0 \end{bmatrix}.$$

For a non-singular square Jacobian,

$$\dot{q} = J^{-1}\xi.$$

More generally, the pseudoinverse is used:

$$\dot{q} = J^\dagger \xi.$$

For a redundant robot, additional joint-space motion may be added without affecting the end-effector task:

$$\dot{q} = J^\dagger \xi + (I - J^\dagger J) z,$$

where the second term lies in the null space of the Jacobian. This null-space freedom can be used to avoid joint limits, improve manipulability, or maintain collision clearance.

9.5 Jacobian Singularities

A kinematic singularity occurs when the Jacobian loses rank:

$$\text{rank}(J) < \min(6, n).$$

At a singularity, some Cartesian velocities cannot be generated through finite joint velocities. Near a singularity, the pseudoinverse may produce excessively large joint commands.

A common regularized alternative is the damped least-squares inverse:

$$J_\lambda^\dagger = J^T (JJ^T + \lambda^2 I)^{-1}.$$

The damping parameter λ trades tracking accuracy for numerical stability.

9.6 Rigid-Body Dynamics

The dynamics of a rigid-link manipulator are commonly written as

$$\boxed{M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + \tau_f = \tau + J^T(q)F_{\text{ext}}.}$$

Here,

- $M(q)$ is the joint-space inertia matrix,
- $C(q, \dot{q})\dot{q}$ contains Coriolis and centrifugal terms,
- $g(q)$ contains gravity torques,
- τ_f represents friction,
- τ is the commanded actuator torque,
- F_{ext} is an external Cartesian wrench.

The term

$$J^T F$$

maps a Cartesian force or wrench into equivalent joint torques.

The inverse dynamics problem asks which torque is required to produce a desired acceleration:

$$\tau = M(q)\ddot{q}_d + C(q, \dot{q})\dot{q} + g(q),$$

ignoring external forces and friction for simplicity.

9.7 PID Control

A proportional–integral–derivative controller operates on tracking error

$$e(t) = x_d(t) - x(t).$$

The control command is

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \dot{e}(t).$$

The proportional term responds to instantaneous error, the derivative term adds damping by responding to error velocity, and the integral term removes persistent steady-state bias.

For joint-space position control,

$$e_q = q_d - q,$$

and a PD controller may take the form

$$\tau = K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}).$$

Gravity compensation can be added:

$$\tau = K_P(q_d - q) + K_D(\dot{q}_d - \dot{q}) + g(q).$$

9.8 Impedance Control

Position control attempts to enforce a desired position. Impedance control instead specifies a desired dynamic relationship between motion error and force.

A Cartesian impedance law can be written as

$$F = K(x_d - x) + D(\dot{x}_d - \dot{x}),$$

where K is a stiffness matrix and D is a damping matrix.

Joint torques are then obtained through

$$\tau = J^T F.$$

A more complete target impedance is

$$M_d(\ddot{x} - \ddot{x}_d) + D_d(\dot{x} - \dot{x}_d) + K_d(x - x_d) = F_{\text{ext}}.$$

The robot therefore behaves approximately like a virtual mass–spring–damper system.

Impedance control is particularly useful during physical interaction because small positional errors produce bounded contact forces rather than an attempt to enforce a perfectly rigid trajectory.

9.9 Operational-Space Control

Operational-space control formulates the control objective directly in task coordinates such as the end-effector position and orientation.

Starting from

$$\dot{x} = J(q)\dot{q},$$

differentiation gives

$$\ddot{x} = J\ddot{q} + \dot{J}\dot{q}.$$

The effective Cartesian inertia is

$$\Lambda(q) = (JM^{-1}J^T)^{-1}.$$

A Cartesian command can then be designed using quantities such as

$$F^* = \Lambda [\ddot{x}_d + K_D(\dot{x}_d - \dot{x}) + K_P(x_d - x)],$$

followed by a mapping into joint torques,

$$\tau = J^T F^* + \text{dynamic compensation terms.}$$

Operational-space control is especially useful for manipulation because many tasks are naturally specified in Cartesian coordinates.

9.10 Model Predictive Control

Model Predictive Control (MPC) repeatedly solves a finite-horizon optimization problem.

Given dynamics

$$x_{t+1} = f(x_t, u_t),$$

MPC solves

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k) + \ell_f(x_H)$$

subject to

$$x_{k+1} = f(x_k, u_k),$$

as well as state and control constraints.

Although an entire sequence

$$u_0, u_1, \dots, u_{H-1}$$

is optimized, only the first control is generally executed. The state is then measured again and the optimization is repeated. This is known as *receding-horizon control*.

MPC is attractive because constraints, dynamics, obstacle avoidance, and future consequences can be represented explicitly.

9.11 Whole-Body Control

Whole-body control coordinates several simultaneous objectives over a robot with many degrees of freedom. For a humanoid or mobile manipulator, objectives may include

- end-effector tracking,
- balance,
- contact-force constraints,
- base motion,
- posture,
- joint-limit avoidance.

Such controllers are often formulated as constrained optimization problems,

$$\min_{\ddot{q}, \tau, F} \sum_i \|J_i \ddot{q} + \dot{J}_i \dot{q} - \ddot{x}_i^*\|_{W_i}^2,$$

subject to rigid-body dynamics and contact constraints.

The weights W_i , or an explicit hierarchy of constraints, determine the relative priority of different tasks.

9.12 Learned Policies and Low-Level Control

In modern robot-learning systems, a learned policy does not necessarily command actuator torques directly. A common architecture separates high-level learned behavior from high-frequency feedback control.

A learned policy may output

$$a_t = x_d,$$

a desired end-effector pose, or

$$a_t = \Delta x,$$

an end-effector displacement, or a short trajectory

$$A_t = [x_t^*, \dots, x_{t+H}^*].$$

A low-level controller then converts these targets into actuator commands:

learned policy $\rightarrow$ desired trajectory $\rightarrow$ impedance/operational-space controller $\rightarrow \tau$.

This separation allows the learned model to operate at a relatively low frequency while a conventional controller maintains stability and reacts quickly to physical disturbances.

10 State Estimation, 3D Perception, and Manipulation

A robot rarely observes the complete physical state required for control. Instead, it receives noisy and incomplete measurements from cameras, inertial sensors, encoders, force sensors, and other devices. State estimation combines these measurements with models of system dynamics to infer hidden quantities such as pose and velocity. Perception extends this process to estimating the geometry and motion of objects in the surrounding environment. Manipulation then uses these estimates to reason about contact, grasping, and physical interaction.

10.1 Bayesian State Estimation

Let

$$x_t$$

denote a hidden system state and

$$z_t$$

a sensor measurement.

A Bayesian filter recursively estimates

$$p(x_t|z_{1:t}, u_{1:t-1}),$$

the posterior distribution over the current state conditioned on all measurements and controls observed so far.

The recursion consists of two stages.

The prediction step uses the system dynamics,

$$p(x_t|z_{1:t-1}) = \int p(x_t|x_{t-1}, u_{t-1})p(x_{t-1}|z_{1:t-1}) dx_{t-1}.$$

The measurement update applies Bayes' rule:

$$p(x_t|z_{1:t}) \propto p(z_t|x_t)p(x_t|z_{1:t-1}).$$

Different filtering algorithms arise from different representations and assumptions about these probability distributions.

10.2 Kalman Filter

The Kalman filter applies to linear systems with Gaussian noise:

$$x_t = A_t x_{t-1} + B_t u_t + w_t,$$

$$z_t = H_t x_t + v_t,$$

with

$$w_t \sim \mathcal{N}(0, Q_t), \quad v_t \sim \mathcal{N}(0, R_t).$$

The prediction equations are

$$\hat{x}_t^- = A_t \hat{x}_{t-1} + B_t u_t,$$

$$P_t^- = A_t P_{t-1} A_t^T + Q_t.$$

The innovation is

$$y_t = z_t - H_t \hat{x}_t^-,$$

with covariance

$$S_t = H_t P_t^- H_t^T + R_t.$$

The Kalman gain is

$$K_t = P_t^- H_t^T S_t^{-1}.$$

The posterior state estimate is

$$\hat{x}_t = \hat{x}_t^- + K_t y_t,$$

and the covariance becomes

$$P_t = (I - K_t H_t) P_t^-.$$

The Kalman gain balances model uncertainty against measurement uncertainty. Accurate measurements receive greater weight, whereas noisy measurements influence the estimate less strongly.

10.3 Extended Kalman Filter

Robotic systems are commonly nonlinear:

$$x_t = f(x_{t-1}, u_t) + w_t,$$

$$z_t = h(x_t) + v_t.$$

The Extended Kalman Filter (EKF) approximates the nonlinear system locally using Jacobians

$$F_t = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}},$$

and

$$H_t = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}}.$$

The state itself is propagated through the nonlinear model,

$$\hat{x}_t^- = f(\hat{x}_{t-1}, u_t),$$

while covariance propagation uses the linearization,

$$P_t^- = F_t P_{t-1} F_t^T + Q_t.$$

The measurement update has the same general structure as the ordinary Kalman filter but uses the nonlinear predicted measurement

$$h(\hat{x}_t^-).$$

The EKF is widely used for combining IMU, wheel encoder, GPS, camera, and other measurements.

10.4 Particle Filters

When the posterior is strongly nonlinear, non-Gaussian, or multimodal, a Gaussian approximation may be inadequate.

A particle filter represents the posterior by weighted samples,

$$p(x_t|z_{1:t}) \approx \sum_{i=1}^N w_t^{(i)} \delta(x_t - x_t^{(i)}).$$

Each particle is propagated through the dynamics, weighted according to measurement likelihood,

$$w_t^{(i)} \propto p(z_t|x_t^{(i)}),$$

and periodically resampled.

Particle filters can represent multiple competing hypotheses but typically require substantially more computation than Kalman-filter-based methods.

10.5 Sensor Fusion and Observability

Sensor fusion combines measurements with complementary strengths.

An IMU provides high-rate acceleration and angular velocity but suffers from integration drift. Cameras provide geometrically informative observations but may operate more slowly and can fail under poor lighting or texture. Encoders provide accurate internal robot configuration but do not independently determine global pose.

A fused estimator exploits these complementary properties.

A state is said to be *observable* if it can, in principle, be inferred from the available sequence of inputs and measurements.

For a linear system

$$x_{t+1} = Ax_t, \quad y_t = Cx_t,$$

the observability matrix is

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}.$$

The system is observable if

$$\text{rank}(\mathcal{O}) = n.$$

Poor observability is a fundamental limitation; no estimator can reliably recover a state variable that does not sufficiently influence the available measurements.

10.6 Camera Geometry

A pinhole camera maps a 3D point

$$P_C = [X \ Y \ Z]^T$$

in the camera frame onto image coordinates

$$u = f_x \frac{X}{Z} + c_x,$$

$$v = f_y \frac{Y}{Z} + c_y.$$

In homogeneous coordinates,

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix},$$

where

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

is the camera intrinsic matrix.

If the point is initially expressed in a world frame,

$$P_C = R_{CW}P_W + t_{CW}.$$

Thus projection combines rigid-body coordinate transformation with perspective projection.

10.7 Perspective- n -Point

The Perspective- n -Point problem, or PnP, estimates a camera pose from known correspondences between 3D points and their observed 2D image locations.

Given

$$P_i \in \mathbb{R}^3$$

and corresponding pixels

$$p_i \in \mathbb{R}^2,$$

the goal is to estimate

$$R, t$$

such that

$$\lambda_i \tilde{p}_i = K(RP_i + t).$$

PnP is widely used for camera localization, fiducial tracking, object pose estimation, and augmented reality.

10.8 Triangulation

Triangulation reconstructs a 3D point from observations in multiple camera views.

Given two known camera projection matrices

$$P_1, \quad P_2$$

and corresponding image points

$$x_1, \quad x_2,$$

the desired 3D point X satisfies

$$x_1 \sim P_1 X, \quad x_2 \sim P_2 X.$$

In ideal geometry, the corresponding camera rays intersect at a single point. Under measurement noise, they generally do not intersect exactly, so triangulation becomes a least-squares problem.

Depth uncertainty grows rapidly when the viewing rays are nearly parallel, which is why stereo systems benefit from a sufficiently large baseline.

10.9 Depth and Optical Flow

Depth represents distance along the camera viewing direction or Euclidean distance from the sensor, depending on convention.

Stereo depth is related approximately to disparity by

$$Z = \frac{fB}{d},$$

where

$$f$$

is focal length,

$$B$$

is camera baseline, and

$$d$$

is pixel disparity.

Optical flow estimates apparent image motion between frames:

$$(u, v) \rightarrow (u + \Delta u, v + \Delta v).$$

Under the brightness constancy assumption,

$$I(x, y, t) \approx I(x + \Delta x, y + \Delta y, t + \Delta t).$$

Linearization gives the optical-flow constraint

$$I_x u + I_y v + I_t = 0.$$

Because this is one equation with two unknown velocity components, additional spatial assumptions are required. This ambiguity is known as the aperture problem.

10.10 Point Clouds

A point cloud represents scene geometry as a collection of three-dimensional points,

$$\mathcal{P} = \{p_1, p_2, \dots, p_N\}, \quad p_i \in \mathbb{R}^3.$$

Additional attributes may include

color, surface normal, intensity, semantic label.

Point clouds may be obtained from stereo cameras, structured-light sensors, time-of-flight cameras, or LiDAR.

Unlike images, point clouds are generally unordered and irregularly sampled, so algorithms must explicitly handle spatial neighborhood structure.

10.11 Iterative Closest Point

Iterative Closest Point (ICP) estimates the rigid transformation aligning two point clouds.

Given source points p_i and corresponding target points q_i , point-to-point ICP solves

$$\min_{R, t} \sum_i \|R p_i + t - q_i\|^2.$$

A commonly used alternative is point-to-plane ICP,

$$\min_{R, t} \sum_i [n_i^T (R p_i + t - q_i)]^2,$$

where n_i is the target surface normal.

ICP alternates between

1. finding correspondences,
2. estimating the transformation,
3. transforming the source points,

until convergence.

Because the optimization is local, ICP generally requires a sufficiently accurate initial alignment.

10.12 Simultaneous Localization and Mapping

SLAM addresses the coupled problem of estimating a robot trajectory and a map from noisy sensor observations.

Given observations

$$z_{1:T}$$

and controls

$$u_{1:T},$$

SLAM seeks a posterior over robot poses and map variables,

$$p(x_{1:T}, m \mid z_{1:T}, u_{1:T}).$$

Modern SLAM systems commonly contain a front end responsible for extracting and associating measurements and a back end responsible for optimizing the resulting geometric constraints.

For pose-graph SLAM, robot poses

$$T_1, \dots, T_N$$

are connected by relative-pose measurements

$$\hat{T}_{ij}.$$

The optimization minimizes residuals of the form

$$e_{ij} = \text{Log} \left(\hat{T}_{ij}^{-1} T_i^{-1} T_j \right),$$

where the logarithm maps an $SE(3)$ transformation into a local vector representation.

Loop closure introduces constraints between locations observed at different times and is essential for reducing accumulated drift.

10.13 Grasping

A grasp is a set of contacts between a robot and an object that permits the robot to constrain or manipulate the object's motion.

A grasp configuration may include

$$G = (T_{EO}, q_{\text{gripper}}),$$

where T_{EO} specifies the gripper pose relative to the object.

A stable grasp depends on properties such as

- contact geometry,
- friction,
- object mass,
- force limits,
- collision clearance,

- task requirements.

Force closure refers to the ability of a set of frictional contacts to resist arbitrary external wrenches within the limits of the contact model.

10.14 Contact and Friction

A simple Coulomb friction model constrains tangential force by

$$\|f_t\| \leq \mu f_n,$$

where

$$f_n$$

is normal force,

$$f_t$$

is tangential force, and

$$\mu$$

is the coefficient of friction.

The admissible set of forces forms a friction cone.

For computational optimization, the friction cone is often approximated using linear inequalities.

Contact constraints are particularly important in tasks such as grasping, pushing, walking, door opening, and bimanual manipulation.

10.15 Force and Torque Sensing

A six-axis force/torque sensor measures a wrench

$$w = \begin{bmatrix} F \\ \tau \end{bmatrix} \in \mathbb{R}^6.$$

Such measurements allow the robot to detect contact, regulate interaction forces, estimate external disturbances, and perform compliant tasks.

Joint torque sensors can similarly be used to estimate external joint torques. Given a sufficiently accurate dynamics model,

$$\tau_{\text{ext}} \approx \tau_{\text{measured}} - \tau_{\text{model}}.$$

Cartesian external forces can then be related to joint torques through

$$\tau_{\text{ext}} = J^T F_{\text{ext}}.$$

10.16 Trajectory Optimization

Trajectory optimization searches for an entire sequence of states and controls that minimizes an objective while satisfying constraints.

A general formulation is

$$\min_{x_{0:T}, u_{0:T-1}} \sum_{t=0}^{T-1} \ell(x_t, u_t) + \ell_f(x_T),$$

subject to

$$x_{t+1} = f(x_t, u_t),$$

as well as constraints such as

$$x_t \in \mathcal{X}, \quad u_t \in \mathcal{U}.$$

The cost may penalize quantities such as

tracking error, control effort, collision proximity, trajectory duration.

Trajectory optimization differs from simple waypoint interpolation because it explicitly incorporates system dynamics and constraints.

10.17 Bimanual Manipulation

Bimanual manipulation involves coordinated control of two manipulators interacting with the same object or task.

If the two end effectors have Jacobians

$$J_L \quad \text{and} \quad J_R,$$

their combined motion may be represented using

$$\begin{bmatrix} v_L \\ v_R \end{bmatrix} = \begin{bmatrix} J_L & 0 \\ 0 & J_R \end{bmatrix} \begin{bmatrix} \dot{q}_L \\ \dot{q}_R \end{bmatrix}.$$

When both hands grasp the same rigid object, their motions are coupled by the object's rigid-body constraints. The controller must therefore coordinate absolute object motion and internal forces between the two contacts.

An important decomposition distinguishes

- *object motion*, which determines the net wrench applied to the object,
- *internal forces*, which alter contact forces without changing the net object wrench.

This distinction is important for carrying large objects, maintaining stable dual-arm grasps, and manipulating articulated or deformable objects.

10.18 Manipulation as Integrated Perception and Control

Complex manipulation tasks require the preceding concepts to operate together.

Consider opening a door. The robot must first estimate the door and handle geometry using perception. It must select a reachable grasp configuration and move the end effector to the handle. After contact, the controller should regulate both motion and force while respecting the door's kinematic constraint.

A simplified processing chain is

camera observations $\rightarrow$ object and handle pose $\rightarrow$ grasp selection $\rightarrow$ trajectory $\rightarrow$ contact $\rightarrow$ force-aware control.

For a deformable object, the problem becomes more difficult because the object state cannot generally be represented by a single rigid transformation in $SE(3)$. The policy must estimate distributed shape or latent state and reason about how contact changes that state.

This illustrates a central principle of modern robotics: state estimation, perception, planning, and control are not independent components. Successful manipulation requires reliable transformation of noisy sensor measurements into state estimates, geometric understanding of the environment, feasible motion generation, and stable feedback control during physical interaction.